

STOCK AND ORDER MANAGEMENT SYSTEM

PROJECT REPORT





Faculty of Computing

Degree program : BSc. in Management Information Systems (Special)
Batch : UGC MIS 21.1
Module : CS 304.3 Advanced Database Management System
Module Lecturer : Mr. Naji Saravanabavan
Group No : 24

Index no	Student name	First name , Last Name	E-mail
22742	R.G.I.S. Senarathna	Isuru Senarathna	rgissenarathna@students.nsbm.ac.lk
22694	C.K.W.D. Mulleriyawa	Chirath Mulleriyawa	ckwdmulleriyawa@students.nsbm.ac.lk
22750	D.P. Jayawardana	Damsara Jayawardana	dpjayawardana@students.nsbm.ac.lk
22727	E.W.S.D. Jayaweera	Sanoj Jayaweera	ewsdjayaweera@students.nsbm.ac.lk
22704	D.C.S. Wanniarachchi	Shashan Wanniarachchi	dcswanniarchchi@students.nsbm.ac.lk
22707	W.K.J.K. Waragoda	Janani Waragoda	wjkwaragoda@students.nsbm.ac.lk
22716	P.A.D.S.S. Pinnawala	Sanudi Pinnawala	padsspinnawala@students.nsbm.ac.lk
22696	W.A.N.R. Dunukewala	Naveen Dunukewala	wanrdunukewala@students.nsbm.ac.lk
22698	T. Jinthuja	Thilagaratnam Jinthuja	tjinthuja@students.nsbm.ac.lk
22703	K.M.G.D.J.M. Dias	Jeewaka Dias	kmgdjmdias@students.nsbm.ac.lk

Acknowledgement

We would like to extend our sincere gratitude to the team at JJ Trading for entrusting us with the opportunity to collaborate on this transformative project. The successful implementation of an advanced database management system for stock and order management is a crucial milestone in the journey to elevate operational efficiency and improve customer service.

Also, we are grateful to extend our heartfelt appreciation to Mr. Naji Saravanabavan, our module lecturer, who provided us with good guidelines for this assignment and for entrusting us with the opportunity to work on this significant project with JJ Trading.

Additionally, we would like to thank our group members for their dedication and contribution to this project and we would like to thank all those who directly or indirectly helped us during this project.

Thank you for your trust and collaboration.

Contents

Preface.....	6
Company Confirmation Letter	7
Project Plan	8
Introduction and the background	9
Introduction to the company	9
Problem Identification.....	10
Scope of the project.....	12
Proposed Solution	13
Objectives of the project	14
Section 01.....	16
Extended Entity Relationship Diagram.....	16
Assumptions:	17
Relational Mapping	18
Data Normalization	19
Data Dictionary	20
Section 02.....	24
Tables and Constrains	24
Database Diagram	27
Section 03.....	28
Trigger Statements	28
User Defined Functions.....	33
Create Views	36
Procedure Statements	38
User Interfaces	45
Application Details.....	45

Screenshots of the Application	46
Analysis and Future Prediction.....	56
GitHub Repository and Backup File Link	60
Conclusion	61

Preface

The Stock and Order Management System project was initiated for JJ Trading to address challenges in their manual stock and order management processes. The company's current operations relied on manual staff member, resulting in issues such as stock counting errors, limited stock logging, and order processing challenges.

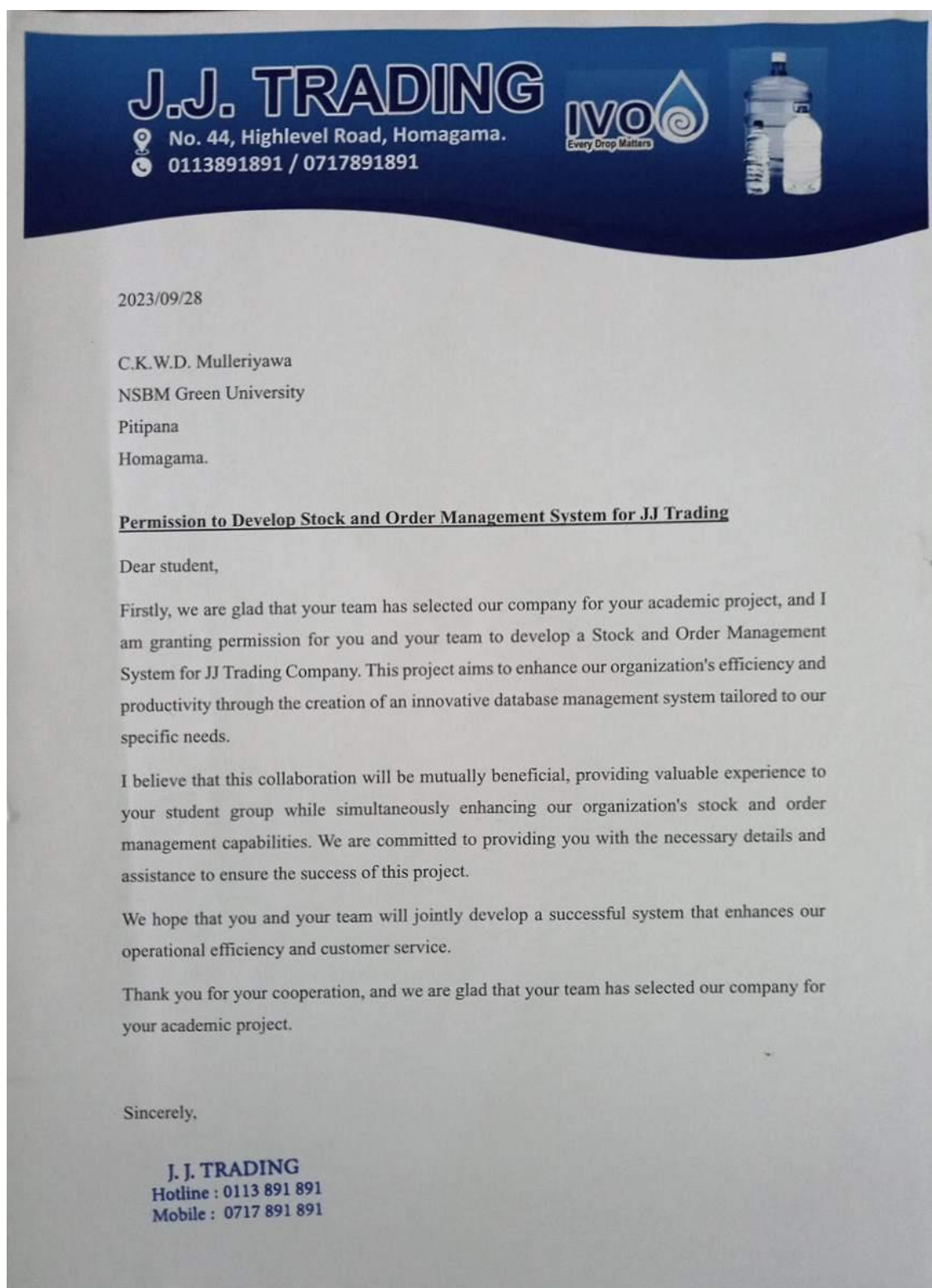
To overcome these issues, we are implementing an advanced database management system that automates stock and order management, enhances data accuracy, and improves data security.

The project team collaborated closely with JJ Trading's management to gain a comprehensive understanding of their specific requirements and operational needs. This report will outline the company background and challenges, objectives of this project, methodology, design, and implementation.

Company Confirmation Letter

Drive Link :

https://drive.google.com/file/d/1Odyn5Uw4qQtqodKcH59FlSXGI_k8DI3U/view?usp=drive_link



Project Plan

- Project Timeline URL :

<https://docs.google.com/spreadsheets/d/1y2ooZeh5erUS3RSsifnCJwvF5JX5LIU7XtsYxA3QTVY/edit?usp=sharing>

- Drive link : https://drive.google.com/file/d/1ENcuEDZf_7DdTwsR7-R-h_aVpElKgm1L/view?usp=drive_link

Stock and Order Management system																																					
Company Name : JJ Trading				Project start : 26/ 09/ 2023																																	
Display week : 04				week 01				week 02				week 03				week 04																					
				26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22						
				M	T	W	T	F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S	S						
TASK				Duration		START DATE		END DATE																													
Project Initiation & Planning																																					
Select organization				1		26/9/2023		26/9/2023																													
Evaluate the company background and getting the company's p				1		27/9/2023		27/9/2023																													
Evaluate current processes and challenges				1		28/9/2023		28/9/2023																													
Define project scope				1		29/9/2023		29/9/2023																													
Define proposed solution and objectives				1		29/9/2023		29/9/2023																													
Create project timelines				1		30/9/2023		30/9/2023																													
Allocating the responsibilities for team members				1		31/9/2023		31/9/2023																													
Designing																																					
Identifying the entities & attributes				1		1/10/2023		1/10/2023																													
Create ER and convert it EERD				1		1/10/2023		1/10/2023																													
Create Relational Mapping				2		2/10/2023		3/10/2023																													
Performing normalization in BCNF				2		4/10/2023		5/10/2023																													
Produce the data dictionary				2		6/10/2023		7/10/2023																													
Developing																																					
Create tables & develop advanced database management system				14		8/10/2023		21/10/2023																													
Project implementation																																					
Data testing				1		21/10/2023		21/10/2023																													
Evaluation								22/10/2023																													

Introduction and the background

Introduction to the company

JJ Trading is a water bottle distribution company located in Homagama city and this company is a small-scale company in this area. They have limited staff members and resources. JJ Trading is dedicated to meeting the essential needs of its customers. The company specializes in the distribution of water bottles to a diverse range of customers, including households, offices, and various events such as weddings, funerals, and other special occasions with special discounts and reasonable prices.

Currently, JJ Trading manages its operations using a system that primarily relies on manual processes. This means that the company predominantly maintains physical records and utilizes manual methods for managing stock, customer information handling, order tracking, managing suppliers, etc. We mainly focus on the **Stock and Order Management Process** of JJ Trading. Currently, JJ Trading does not have dedicated software or automated system in place for managing its stock.

JJ Trading is committed to providing efficient and reliable water bottle distribution services. They ensure that water bottles are procured, stored, and delivered promptly to fulfill the demands of their customers. The company aims to establish a reputation for quality service and customer satisfaction in the local market while also seeking opportunities to expand its customer base.

To elevate operational efficiency, streamline stock management, and enhance customer service, JJ Trading is poised to embark on a transformative journey. The introduction of an advanced database management system for stock and order management will revolutionize their operations.

With this technological enhancement, JJ Trading anticipates a marked improvement in service quality, customer satisfaction, and a promising trajectory of business growth within the competitive landscape of water bottle distribution in Homagama City.

Company visits video link :

https://drive.google.com/file/d/1QPzIRrpfIVKw93ZIE1HkGg5o_34QiqYt/view?usp=drive_link

Problem Identification

We conducted an in-person interview with the owner and employees to gather information about the current business process of the JJ Trading Company. we emphasized despite their long-standing presence in the market, they have encountered several problems in their current stock and order management process.

1. Manual Stock Counting:

- The current stock and order management process at JJ Trading is currently handled manually. Each morning, one of the staff members must count and record the number of water bottles in the shop, which is then displayed on a board.
- This manual process is not only time-consuming but also prone to errors, leading to inaccuracies in stock levels.

2. Order Processing Challenges:

- The current order processing method is manual, where staff manually handle customer orders.
- This process can be error-prone, as it relies on individuals to match customer orders with the available stock.
- The lack of automation can result in delays and potential order mismatches.

3. Challenges in Obtaining Predictions and Analytical Data for Stock:

- The current process makes it difficult to accurately forecast stock requirements and obtain data-driven insights for stock management.
- It can lead to problems such as stockouts, overstocking, inefficient reordering, and reduced profit margins.

4. Challenges in Categorizing Bottles Due to Limited Stock Space:

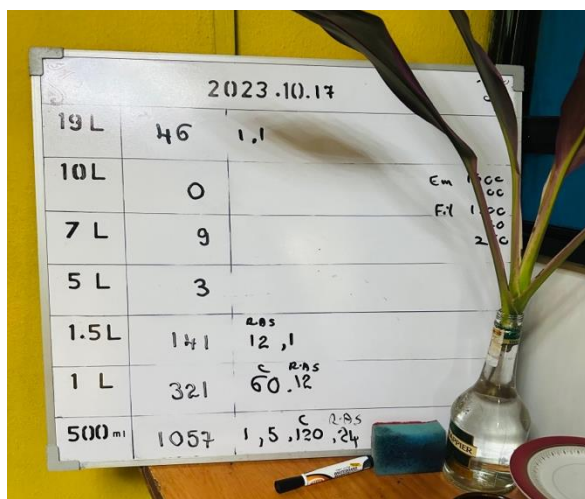
- Currently, they have limited storage space for their water bottle stock makes it challenging to efficiently organize and categorize products.
- It can result in disorganization, inefficient picking, increased risk of stock loss and damage, and limited visibility within their stock area.

5. Data Security Concerns:

- Storing sensitive customer and operational data manually can expose the company to data security risks and unauthorized access.

6. Data Inaccuracy:

- The reliance on manual stock counting and order processing increases the risk of data inaccuracies.
- Data is recorded using pen and paper, which can lead to inconsistencies, financial losses, and confusion among employees and customers due to unreliable information.



Scope of the project

The scope of the project for JJ Trading is to implement an advanced database management system for stock and order management to enhance operational efficiency and improve customer service.

This project aims to modernize the company's stock and order management practices, which currently heavily rely on manual processes.

- **In scope :**

1. Implementation of an advanced database management system for stock and order management.
2. Automating stock counting processes to eliminate manual counting.
3. Automating the order processing to reduce manual handling and improve efficiency.
4. Comprehensive stock logging to record and analyze the history of stock movements.
5. Implementing data security measures to protect sensitive customer and operational data.

- **Out of scope :**

1. Employee training in using the new database management system.
2. Customization of the database management system for purposes beyond stock and order management.
3. Marketing and promotion activities related to the project.

Proposed Solution

Our proposed solution for JJ Trading is an all-inclusive stock and order management system, designed to enhance operational efficiency and streamline stock and order management processes. They have limited resources, and this company is not a large-scale company (currently they are new to the business industry), they have limited staff and because of that, they will hope to use this system in one computer.

For all these reasons we decided to develop a **Standalone Application** for this stock and order management process of the JJ Trading company.



Objectives of the project

The primary objectives of the project are to address the challenges and issues JJ Trading faces in its stock and order management process.

By implementing an advanced database management system, the project aims to achieve the following goals:

1. Implement an Automated Stock Management:

- The foremost objective of the project is to introduce an advanced database management system for stock management.
- By doing so, we aim to eliminate the time-consuming and error-prone manual stock-counting process of JJ Trading.

2. Enhanced Order Processing:

- Automate the order management process to reduce manual handling.
- This will include an order management feature that matches customer orders with available stock and allows for efficient order fulfillment.

3. Comprehensive Stock Logging:

- We will develop a comprehensive stock log within the database system to track stock movements, aiding in better decision-making, trend analysis, getting predictions, and optimizing stock levels.

4. Implementing a systematic water bottle categorization of this system:

- Implementing systematic categorization helps to optimize stock space, reduce disorganization, improve product accessibility, and enhance customer service.

5. Supplier Management:

- Facilitate efficient supplier information management, improving relationships and negotiations.

6. Customer Service Improvement:

- Enhancing customer satisfaction by providing real-time stock availability information, ensuring timely order fulfillment.

7. Improve Data Accuracy:

- Our project seeks to eliminate data inaccuracies that arise from manual stock counting and paper-based order processing of JJ Trading.
- Accurate data is essential for financial stability and ensuring that employees and customers have confidence in the information provided.

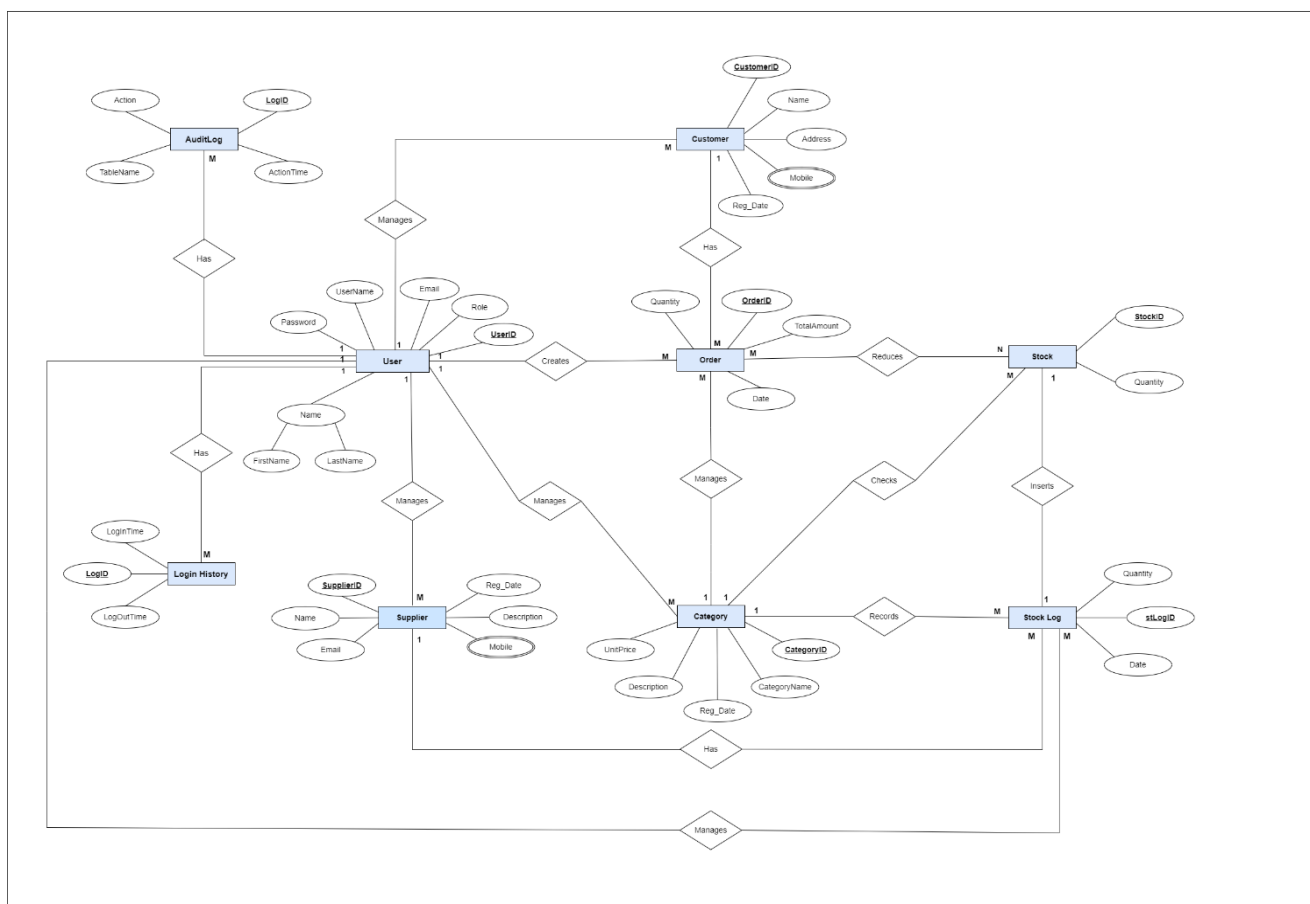
Section 01

Extended Entity Relationship Diagram

- Drive link :

https://drive.google.com/file/d/1tHdU63dFeG4-U4Z6byS2rEOE-9sP61zO/view?usp=drive_link

EER Diagram

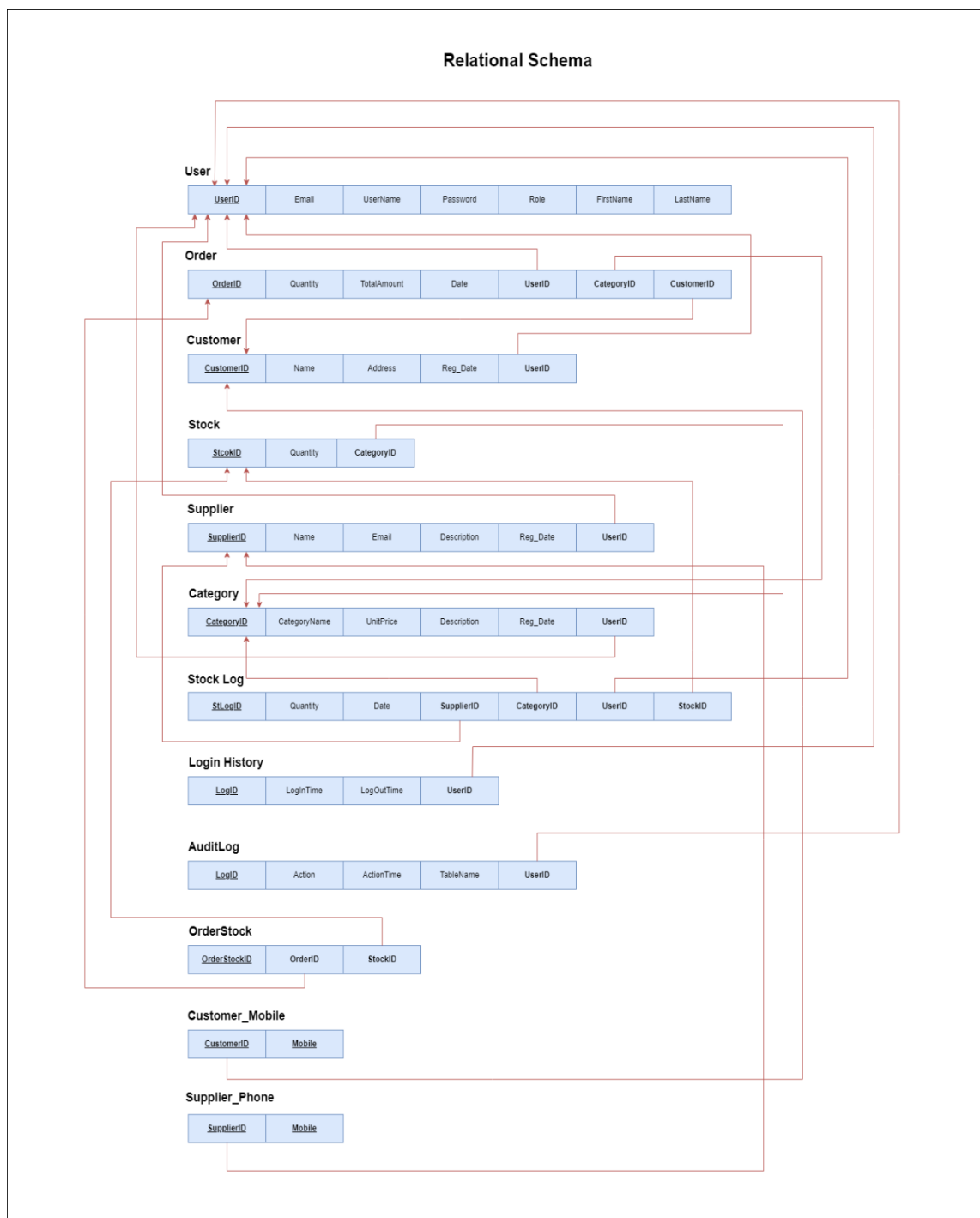


Assumptions:

- One User can create multiple Orders. In other words, many Orders can be created by a single User : (1:M)
- One User can manage multiple Customers. In other words, many Customers can be managed by a single User : (1:M)
- Multiple Orders are associated with one Customer. In other words, for one Customer there can be multiple Orders : (M:1)
- Multiple Orders reduces Multiple Stocks. In other words, for many stocks has many orders : (M:N)
- One User can manage Many Suppliers. In other words, multiple Suppliers can be managed by one User : (1:M)
- One Supplier has Many Stock Logs associated with their transactions : (1:M)
- One User can manage multiple Stock Logs : (1:M)
- One User can manage multiple Categories : (1:M)
- One User has many Login Histories : (1:M)
- One User has many Audit Logs : (1:M)
- One Category can be associated with multiple Stock logs : (1:M)
- Many Stocks can be checked by One Category, means that multiple Stocks can be categorized under a single Category : (M:1)
- One Stock inserts One Stock Log, where each Stock insertion is associated with one specific Stock Log : (1:1)
- “User” can be either “Super Admin” or a “Staff - User”.
- The “Role” attribute in the “User” entity, decides whether the user should be redirected to the User dashboard or Super Admin dashboard.
- Only Super Admin can manage users and view analytical data.
- Multiple mobile numbers are stored for a particular Supplier.
- Multiple mobile number is stored for a particular Customer.
- The Attribute named “Action” stores the user interactions like insert, update, and delete for a particular Stock Log.
- Though there are multiple Suppliers, water bottles are sold under same name that is “JJBottleStage”

Relational Mapping

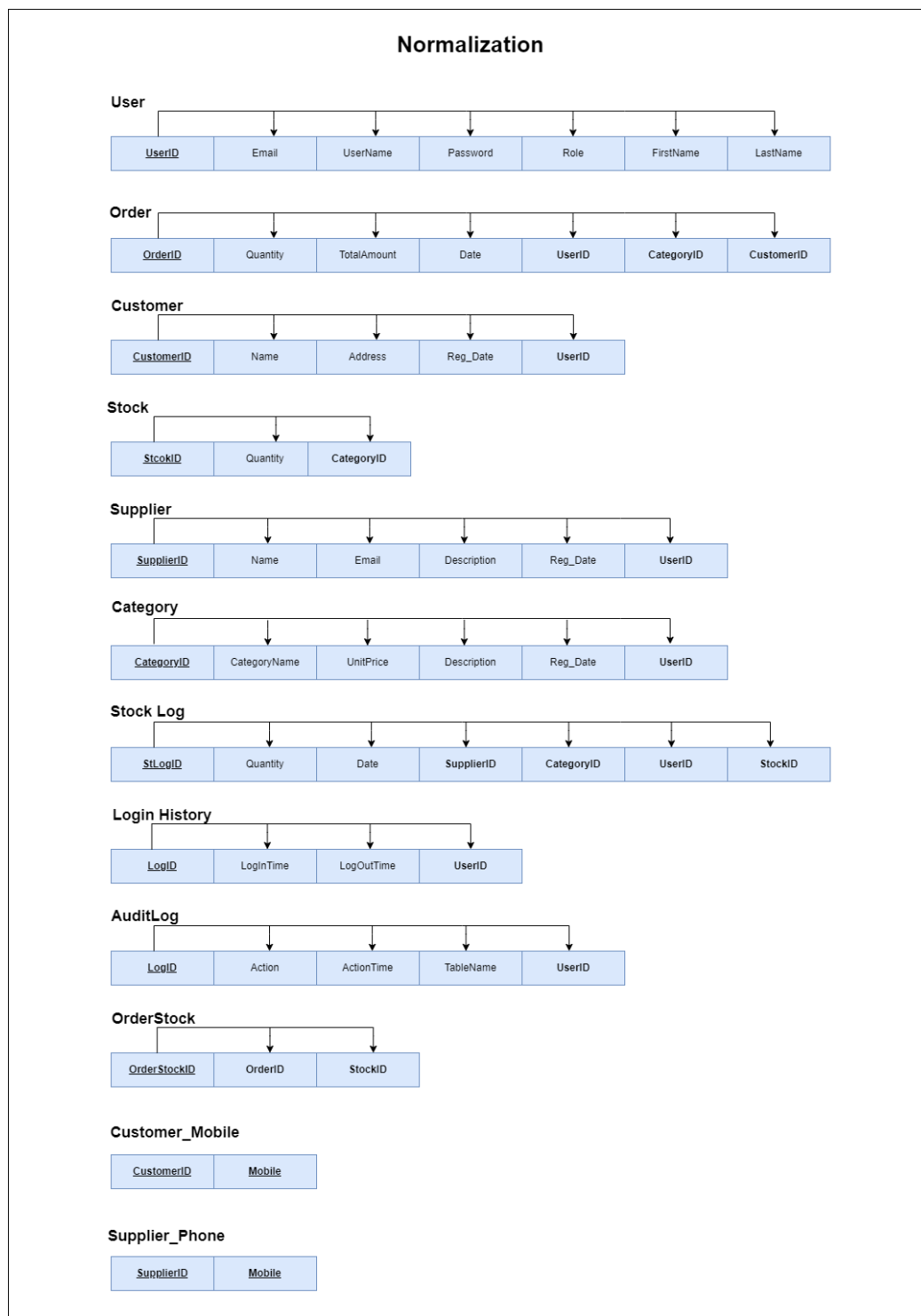
- Drive link : https://drive.google.com/file/d/1pY_8fKH9Hdat7-jFNN97ZMZcNYAWSkfB/view?usp=drive_link



Data Normalization

- Drive link : https://drive.google.com/file/d/1JSr5m_eGnE8Cvguur14fiWN848eji0M9/view?usp=drive_link

Note : All tables in the relational schema are assumed to already be in 3rd normal form.



Data Dictionary

Main Table Name	Field Name	Data Type	Description
User Table	<u>UserID</u>	INT	A unique identification number of the user
	UserName	VARCHAR(255)	The user's chosen username for system access
	Password	VARCHAR(255)	The confidential password associated with the user's account
	Email	VARCHAR(255)	The email address of the user
	Role	VARCHAR(255)	The job role of the user
	FirstName	VARCHAR(255)	The user's first name of the user
	LastName	VARCHAR(255)	The user's last name of the user
Login History Table	<u>LogID</u>	INT	A unique number for each login/logout record
	LoginTime	DATETIME	The time when a user login
	LogoutTime	DATETIME	The time when a user logs out

Category Table	<u>CategoryID</u>	INT	▪ A unique identification number of each category
	CategoryName	VARCHAR(255)	▪ The Name of the category
	Reg_Date	DATE	▪ Date of customer registered
	Description	TEXT	▪ The description of each category
	UnitPrice	DECIMAL(10,2)	▪ The unit price of each category
Order Table	<u>OrderID</u>	INT	▪ A unique identification number of the customer order
	Date	DATE	▪ Date of order create
	Quantity	INT	▪ Quantity of the order
	TotalAmount	DECIMAL(10,2)	▪ The total amount of the order
OrderStock Table	<u>OrderStockID</u>	INT	▪ A unique identification number of the orderstock

Stock Table	<u>StockID</u>	INT	▪ A unique identification number of the stock
	Quantity	INT	▪ Available quantity of the stock
Stock Log Table	<u>StLogID</u>	INT	▪ A unique identification number of the stock log
	Date	DATE	▪ Date of the stored
Supplier Table	<u>SupplierID</u>	INT	▪ A unique identification number of the supplier
	Name	VARCHAR(255)	▪ The name of the supplier
	Description	TEXT	▪ The description of the supplier
	Email	VARCHAR(255)	▪ The email address of the supplier
	Reg_Date	DATE	▪ Date of Supplier registered
Supplier_Phone Table	<u>SupplierID</u>	INT	▪ A unique identification number of the supplier
	Mobile	VARCHAR(255)	▪ The supplier mobile numbers (multi-valued attribute)

Customer Table	<u>CustomerID</u>	INT	▪ A unique identification number of the customer
	Name	VARCHAR(255)	▪ The name of the customer
	Reg_Date	DATE	▪ Date of customer registered
	Adress	VARCHAR(255)	▪ The address of the customer (single-valued attribute)
Customer_Mobile Table	<u>CustomerID</u>	INT	▪ A unique identification number of the customer
	Mobile	VARCHAR(255)	▪ The customer mobile numbers (multi-valued attribute)
AuditLog Table	<u>LogID</u>	INT	▪ A unique identification number of the audit log
	TableName	VARCHAR(255)	▪ The name of the table
	Action	VARCHAR(255)	▪ The user interactions like insert, update, and deleting.
	ActionTime	DATETIME	▪ The action date and time

Section 02

Tables and Constrains

```
create database JJBottleStage;
```

```
use JJBottleStage;
```

```
CREATE TABLE Users (  
    UserID INT NOT NULL PRIMARY KEY,  
    UserName VARCHAR(255) NOT NULL,  
    Password VARCHAR(255) NOT NULL,  
    Email VARCHAR(255),  
    Role VARCHAR(255),  
    FirstName VARCHAR(255),  
    LastName VARCHAR(255),  

```

```
);
```

```
CREATE TABLE LoginHistory (  
    LogID INT IDENTITY(1, 1) PRIMARY KEY,  
    UserID INT,  
    LoginTime DATETIME,  
    LogoutTime DATETIME,  
    FOREIGN KEY (UserID) REFERENCES Users(UserID)  

```

```
);
```

```
CREATE TABLE Category (  
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,  
    CategoryName VARCHAR(255) NOT NULL,  
    Reg_Date DATE NOT NULL,  
    Description TEXT,  
    UnitPrice DECIMAL(10, 2),  
    UserID INT NOT NULL,  
    FOREIGN KEY (UserID) REFERENCES Users(UserID),  

```

```
);
```

```
CREATE TABLE Orders (
    OrderID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT NOT NULL,
    Date DATE NOT NULL,
    Quantity INT NOT NULL,
    TotalAmount DECIMAL(10, 2),
    CustomerID INT NOT NULL,
    CategoryID INT NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
```

```
CREATE TABLE OrderStock (
    OrderStockID INT IDENTITY(1,1) PRIMARY KEY,
    OrderID INT,
    StockID INT,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (StockID) REFERENCES Stock(StockID)
);
```

```
CREATE TABLE Stock (
    StockID INT IDENTITY(1,1) PRIMARY KEY,
    Quantity INT NOT NULL,
    CategoryID INT NOT NULL,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
```

```
CREATE TABLE Stock_Log (
    StLogID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT,
    Date DATE,
    CategoryID INT,
    StockID INT,
    Quantity INT,
    SupplierID INT NOT NULL,
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (StockID) REFERENCES Stock(StockID)
);
```

```
CREATE TABLE Supplier (
    SupplierID INT IDENTITY(1,1) PRIMARY KEY,
    Name VARCHAR(255) NOT NULL,
    Description TEXT,
    Email VARCHAR(255),
    Reg_Date DATE NOT NULL,
    UserID INT NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID),

);
```

```
CREATE TABLE Supplier_Mobile (
    SupplierID INT,
    Mobile VARCHAR(20),
    PRIMARY KEY (SupplierID, Number),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);
```

```
CREATE TABLE Customer (
    CustomerID INT NOT NULL PRIMARY KEY,
    Name VARCHAR(255) NOT NULL,
    Reg_Date DATE NOT NULL,
    Address VARCHAR(255),
    UserID INT NOT NULL,

    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
```

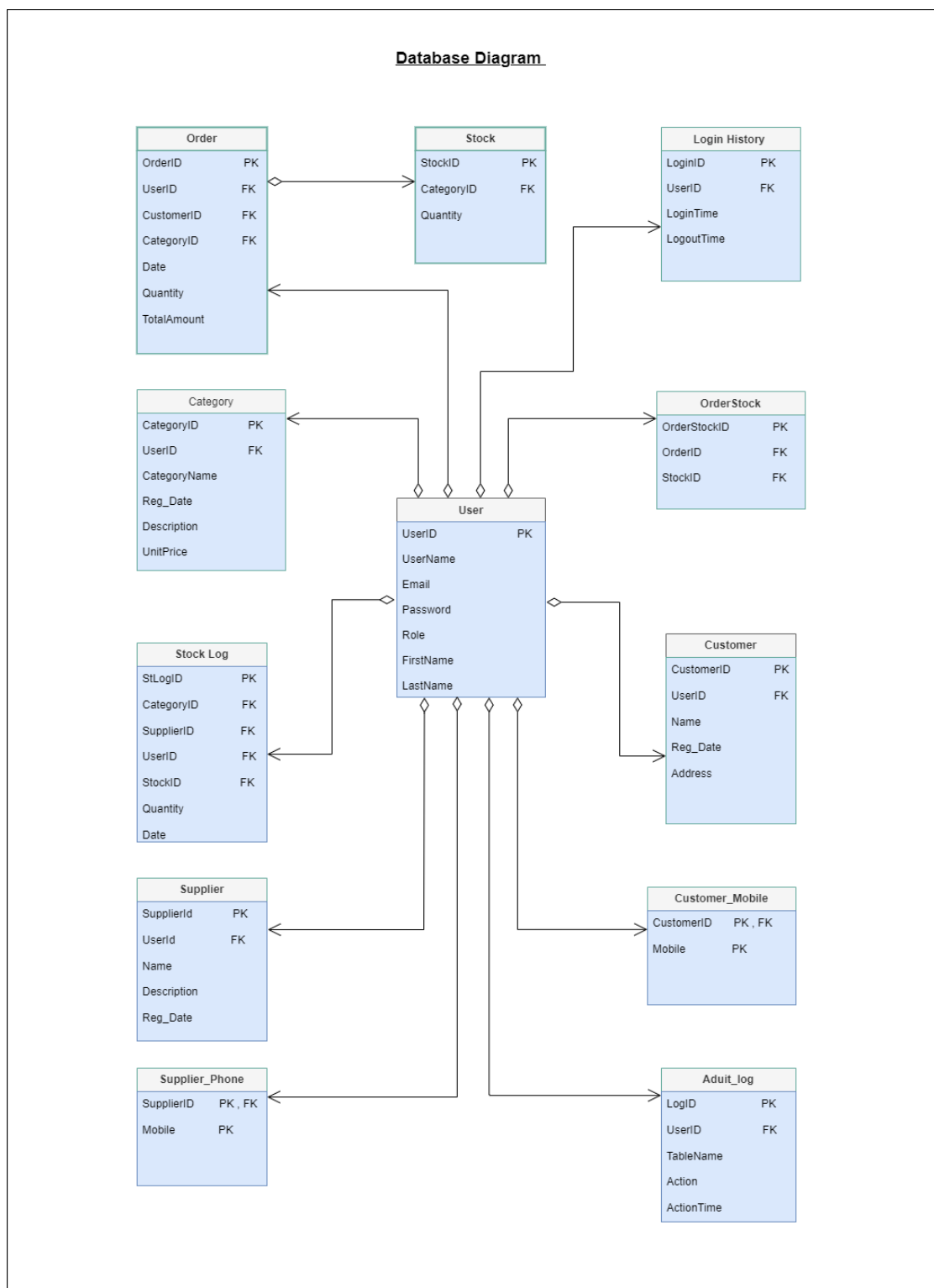
```
CREATE TABLE Customer_Mobile (
    CustomerID INT,
    Mobile VARCHAR(20),
    PRIMARY KEY (CustomerID, Mobile),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
```

```
CREATE TABLE AuditLog (
    LogID INT IDENTITY(1,1) PRIMARY KEY,
    TableName VARCHAR(255),
    Action VARCHAR(10),
    UserID INT,
    ActionTime DATETIME
);
```

Database Diagram

- Drive link :

https://drive.google.com/file/d/1YgoUgx7jCf7NCpC7CHYS1Ek7u7eXGDHm/view?usp=drive_link



Section 03

Trigger Statements

- **Trigger :** Category_CheckConstraints

```
CREATE TRIGGER Category_CheckConstraints
ON Category
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT CategoryName
        FROM INSERTED AS I
        WHERE I.CategoryName IN (
            SELECT C.CategoryName
            FROM Category AS C
            WHERE C.CategoryID NOT IN (SELECT CategoryID FROM INSERTED)
        )
    )
    BEGIN
        PRINT ('Category names must be unique.');
```

```
        ROLLBACK;
        RETURN;
    END
    IF EXISTS (
        SELECT CategoryName
        FROM INSERTED AS I
        WHERE NOT (I.CategoryName LIKE '%L' OR I.CategoryName LIKE '%ML')
```

```
    )
    BEGIN
        PRINT ('Category names must end with ''L'' or ''ML''.');
```

```
        ROLLBACK;
    END
END;
```

- **Trigger : Category_AuditTrigger**

```
CREATE TRIGGER Category_AuditTrigger
ON Category
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Action NVARCHAR(10);

    IF EXISTS (SELECT * FROM INSERTED)
        SET @Action = 'INSERT';
    ELSE IF EXISTS (SELECT * FROM DELETED)
        SET @Action = 'DELETE';
    ELSE
        SET @Action = 'UPDATE';

    INSERT INTO AuditLog (TableName, Action, UserID, ActionTime)
    SELECT 'Category', @Action, (SELECT ISNULL(UserID, 0) FROM INSERTED) AS
    UserID, GETDATE();
END;
```

- **Trigger : Stock_Log_AuditTrigger**

```
CREATE TRIGGER Stock_Log_AuditTrigger
ON Stock_Log
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Action NVARCHAR(10);

    IF EXISTS (SELECT * FROM INSERTED)
        SET @Action = 'INSERT';
    ELSE IF EXISTS (SELECT * FROM DELETED)
        SET @Action = 'DELETE';
    ELSE
        SET @Action = 'UPDATE';

    INSERT INTO AuditLog (TableName, Action, UserID, ActionTime)
    SELECT 'Stock_Log', @Action, (SELECT ISNULL(UserID, 0) FROM INSERTED) AS
    UserID, GETDATE();
END;
```

- **Trigger : CustomerMOBILEvalidation**

```
CREATE TRIGGER CustomerMOBILEvalidation
ON Customer_Mobile
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Mobile VARCHAR(10);
    SELECT @Mobile = Mobile
    FROM INSERTED;
    IF LEN(@Mobile) <> 10
    BEGIN
        PRINT('Mobile number and Customer ID must contain 10 digits.');
```

```
        ROLLBACK;
```

```
    END
```

```
END;
```

- **Trigger : SupplierMOBILEvalidation**

```
CREATE TRIGGER SupplierMOBILEvalidation
ON Supplier_Mobile
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Mobile VARCHAR(10);
    SELECT @Mobile = Mobile
    FROM INSERTED;
    IF LEN(@Mobile) <> 10
    BEGIN
        PRINT('Mobile number must contain 10 digits.');
```

```
        ROLLBACK;
```

```
    END
```

```
END;
```

- **Trigger : DeleteCustomerMobile**

```
CREATE TRIGGER DeleteCustomerMobile
ON Customer
AFTER DELETE
AS
BEGIN
    DELETE FROM Customer_Mobile
    WHERE CustomerID IN (SELECT deleted.CustomerID FROM deleted);
END;
```

- **Trigger : DeleteSupplierMobile**

```
CREATE TRIGGER DeleteSupplierMobile
ON Supplier
AFTER DELETE
AS
BEGIN
    DELETE FROM Supplier_Mobile
    WHERE SupplierID IN (SELECT deleted.SupplierID FROM deleted);
END;
```

- **Trigger : Customer_AuditTrigger**

```
CREATE TRIGGER Customer_AuditTrigger
ON Customer
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Action NVARCHAR(10);

    IF EXISTS (SELECT * FROM INSERTED)
        SET @Action = 'INSERT';
    ELSE IF EXISTS (SELECT * FROM DELETED)
        SET @Action = 'DELETE';
    ELSE
        SET @Action = 'UPDATE';

    INSERT INTO AuditLog (TableName, Action, UserID, ActionTime)
    SELECT 'Customer', @Action, (SELECT ISNULL(UserID, 0) FROM INSERTED) AS
    UserID, GETDATE();
END;
```

- **Trigger : Suplyer_AuditTrigger**

```
CREATE TRIGGER Suplyer_AuditTrigger
ON Supplier
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Action NVARCHAR(10);
    IF EXISTS (SELECT * FROM INSERTED)
        SET @Action = 'INSERT';
    ELSE IF EXISTS (SELECT * FROM DELETED)
        SET @Action = 'DELETE';
    ELSE
        SET @Action = 'UPDATE';
    INSERT INTO AuditLog (TableName, Action, UserID, ActionTime)
    SELECT 'Supplier', @Action, (SELECT ISNULL(UserID, 0) FROM INSERTED) AS
    UserID, GETDATE();
END;
```

User Defined Functions

- **Function :** CheckStockQuantity()

```
CREATE FUNCTION CheckStockQuantity()
RETURNS NVARCHAR(255)
AS
BEGIN
    DECLARE @LowStockThreshold INT = 50;
    DECLARE @HighStockThreshold INT = 2000;
    DECLARE @AlertMessage NVARCHAR(255);

    IF (SELECT COUNT(*) FROM Stock WHERE Quantity < @LowStockThreshold) > 0
    BEGIN
        SET @AlertMessage = 'Low stock alert: Some items have less than ' +
        CAST(@LowStockThreshold AS NVARCHAR(10)) + ' quantity.';
    END

    ELSE IF (SELECT COUNT(*) FROM Stock WHERE Quantity > @HighStockThreshold) >
    0
    BEGIN
        SET @AlertMessage = 'High stock alert: Some items have more than ' +
        CAST(@HighStockThreshold AS NVARCHAR(10)) + ' quantity.';
    END
    ELSE
    BEGIN
        SET @AlertMessage = 'Stock levels are within the normal range.';
    END

    RETURN @AlertMessage;
END;
```

- **Function :** GetStockLogDataWithNames()

```
CREATE FUNCTION GetStockLogDataWithNames()
RETURNS TABLE
AS
RETURN
(
    SELECT SL.StLogID, SL.Date, C.CategoryName,
    SU.Name AS SupplierName, SU.SupplierID, SL.Quantity AS StockLogQuantity
    FROM Stock_Log SL
    JOIN Category C ON SL.CategoryID = C.CategoryID
    JOIN Supplier SU ON SL.SupplierID = SU.SupplierID
);
```

- **Function :** GetStockPercentageByCategory()

```
CREATE FUNCTION dbo.GetStockPercentageByCategory()
RETURNS TABLE
AS
RETURN
(
    SELECT
        c.CategoryName,
        s.Quantity AS StockQuantity,
        (s.Quantity * 100.0) / SUM(s.Quantity) OVER () AS StockPercentage
    FROM
        Category c
    JOIN
        Stock s ON c.CategoryID = s.CategoryID
);
```

- **Function :** CalculateDailyTotalProfit(@Date DATE)

```
CREATE FUNCTION CalculateDailyTotalProfit(@Date DATE)
RETURNS DECIMAL(10, 2)
AS
BEGIN
    DECLARE @TotalProfit DECIMAL(10, 2);

    SELECT @TotalProfit = SUM(TotalAmount)
    FROM Orders
    WHERE Date = @Date;

    RETURN @TotalProfit;
END;
```

- **Function :** CalculateDailyTotalProfitAndOrders(@Date DATE)

```
CREATE FUNCTION CalculateDailyTotalProfitAndOrders(@Date DATE)
RETURNS TABLE
AS
RETURN (
    SELECT
        SUM(TotalAmount) AS TotalProfit,
        COUNT(OrderID) AS TotalOrders
    FROM Orders
    WHERE Date = @Date
);
```

- **Function :** CalculateDailyOrderPercentage(@Date DATE)

```
CREATE FUNCTION CalculateDailyOrderPercentage(@Date DATE)
RETURNS DECIMAL(5, 2)
AS
BEGIN
    DECLARE @OrderCount INT;

    SELECT @OrderCount = COUNT(*)
    FROM Orders
    WHERE Date = @Date;

    DECLARE @Percentage DECIMAL(5, 2);
    IF @OrderCount = 0
    BEGIN
        SET @Percentage = 0;.
    END
    ELSE
    BEGIN
        SET @Percentage = (@OrderCount * 100.0) / 50;
    END

    RETURN @Percentage;
END;
```

Create Views

- **View : UserLoginHistoryView**

```
CREATE VIEW UserLoginHistoryView AS
SELECT
    U.UserID,
    U.UserName,
    U.Email,
    U.Role,
    CONCAT(U.FirstName, ' ', U.LastName) AS FirstName,
    MAX(LH.LoginTime) AS LastLoginTime,
    MAX(LH.LogoutTime) AS LastLogoutTime
FROM
    Users U
LEFT JOIN
    LoginHistory LH
ON
    U.UserID = LH.UserID
GROUP BY
    U.UserID, U.UserName, U.Email, U.Role, U.FirstName, U.LastName;
```

- **View : StockLogDetailsView**

```
CREATE VIEW StockLogDetailsView AS
SELECT
    SL.StLogID,
    C.CategoryName,
    S.Name AS SupplierName,
    U.FirstName AS ModifiedBy,
    AL.Action AS ModificationAction,
    AL.ActionTime AS ModifiedDate,
    SL.Quantity,
    SL.Date AS StockLogDate
FROM
    Stock_Log SL
INNER JOIN
    Category C ON SL.CategoryID = C.CategoryID
INNER JOIN
    Supplier S ON SL.SupplierID = S.SupplierID
INNER JOIN
    Users U ON SL.UserID = U.UserID
LEFT JOIN
    AuditLog AL ON SL.StLogID = AL.LogID;
```

- **View : OrderDetails**

```
CREATE VIEW OrderDetails AS
SELECT
    o.OrderID,
    o.UserID,
    o.Date,
    o.Quantity,
    o.TotalAmount,
    o.CustomerID,
    o.CategoryID,
    c.CategoryName,
    cu.Name AS CustomerName,
    cu.Address AS CustomerAddress
FROM Orders o
INNER JOIN Category c ON o.CategoryID = c.CategoryID
INNER JOIN Customer cu ON o.CustomerID = cu.CustomerID;
```

- **View : SupplierDetailsView**

```
CREATE VIEW SupplierDetailsView AS
SELECT
    S.SupplierID,
    S.Name AS SupplierName,
    S.Description,
    S.Email,
    S.Reg_Date,
    SM.Mobile AS SupplierMobile
FROM
    Supplier AS S
LEFT JOIN
    Supplier_Mobile AS SM
ON
    S.SupplierID = SM.SupplierID;
```

Procedure Statements

- **Procedure : InsertLoginHistory**

```
CREATE OR ALTER PROCEDURE [dbo].[InsertLoginHistory]
    @UserID INT,
    @LoginTime DATETIME
AS
BEGIN
    INSERT INTO LoginHistory (UserID, LoginTime)
    VALUES (@UserID, @LoginTime);
END;
```

- **Procedure : RecordLogoutTime**

```
CREATE OR ALTER PROCEDURE [dbo].[RecordLogoutTime] (
    @UserID INT
)
AS
BEGIN
    DECLARE @LoginHistoryID INT;

    SELECT TOP 1 @LoginHistoryID = LogID
    FROM LoginHistory
    WHERE UserID = @UserID
    AND LogoutTime IS NULL
    ORDER BY LogID DESC;

    IF @LoginHistoryID IS NOT NULL
    BEGIN
        UPDATE LoginHistory
        SET LogoutTime = GETDATE()
        WHERE LogID = @LoginHistoryID;
    END;
END;
```

- **Procedure :** InsertOrUpdateStockData (For storing data to stock while updating the stocklog)

```

CREATE PROCEDURE InsertOrUpdateStockData
    @CategoryName VARCHAR(255),
    @QuantityChange INT,
    @SupplierID INT,
    @UserID INT
AS
BEGIN
    BEGIN TRANSACTION;
    DECLARE @CategoryID INT;
    DECLARE @StockID INT;
    IF NOT EXISTS (SELECT 1 FROM Supplier WHERE SupplierID = @SupplierID)
    BEGIN
        ROLLBACK;
        PRINT('Supplier with SupplierID ' + CAST(@SupplierID AS VARCHAR(10)) + '
does not exist. Please save the supplier first.');
```

RETURN;

```

    END
    SELECT @CategoryID = CategoryID
    FROM Category
    WHERE CategoryName = @CategoryName;

    IF @CategoryID IS NOT NULL
    BEGIN
        SELECT @StockID = StockID
        FROM Stock
        WHERE CategoryID = @CategoryID;

        IF @StockID IS NOT NULL
        BEGIN
            UPDATE Stock
            SET Quantity = Quantity + @QuantityChange
            WHERE StockID = @StockID;
        END
        ELSE
        BEGIN
            INSERT INTO Stock (Quantity, CategoryID)
            VALUES (@QuantityChange, @CategoryID);

            SET @StockID = SCOPE_IDENTITY();
        END

        INSERT INTO Stock_Log (Date, CategoryID, StockID, SupplierID, UserID,
Quantity)
        VALUES (GETDATE(), @CategoryID, @StockID, @SupplierID, @UserID,
@QuantityChange);

        COMMIT;
    END
    ELSE
    BEGIN
        ROLLBACK;
        PRINT('CategoryName not found. Please select a valid category.');
```

END

```

    END;

```

▪ **Procedure : UpdateStockLog**

```

CREATE PROCEDURE UpdateStockLog
    @StLogID INT,
    @CategoryName VARCHAR(255),
    @SupplierID INT,
    @NewQuantity INT,
    @Action CHAR(1)
AS
BEGIN
    BEGIN TRANSACTION;
    IF NOT EXISTS (SELECT 1 FROM Supplier WHERE SupplierID = @SupplierID)
    BEGIN
        ROLLBACK;
        THROW 51000, 'Supplier with does not exist. Please save the supplier
first.', 1;
        RETURN;
    END
    DECLARE @CategoryID INT;
    SELECT @CategoryID = CategoryID
    FROM Category
    WHERE CategoryName = @CategoryName;

    IF @CategoryID IS NULL
    BEGIN
        ROLLBACK;
        THROW 51000, 'Category does not exist. Please select a valid category.', 1;
        RETURN;
    END
    DECLARE @StockID INT;
    SELECT @StockID = StockID
    FROM Stock
    WHERE CategoryID = @CategoryID;

    IF @StockID IS NULL
    BEGIN
        ROLLBACK;
        THROW 51000, 'Stock ID does not exist for the selected category. Please
check your data.', 1;
        RETURN;
    END
    DECLARE @CurrentQuantity INT;
    SELECT @CurrentQuantity = Quantity
    FROM Stock_Log
    WHERE StLogID = @StLogID;

    IF @Action = 'A'
    BEGIN
        SET @NewQuantity = @CurrentQuantity + @NewQuantity;
    END
    ELSE IF @Action = 'R'

```

```

BEGIN
    SET @NewQuantity = @CurrentQuantity - ABS(@NewQuantity);
END

UPDATE Stock_Log
SET CategoryID = @CategoryID, SupplierID = @SupplierID, Quantity =
@NewQuantity
WHERE StLogID = @StLogID;
UPDATE Stock
SET Quantity = Quantity + (@NewQuantity - @CurrentQuantity)
WHERE StockID = @StockID;

COMMIT;
END

```

▪ **Procedure : DeleteStockLogAndReduceStock**

```

CREATE PROCEDURE DeleteStockLogAndReduceStock
@StLogID INT
AS
BEGIN
    BEGIN TRANSACTION;

    DECLARE @CategoryID INT;
    DECLARE @StockID INT;
    DECLARE @QuantityToDelete INT;

    SELECT @CategoryID = SL.CategoryID, @StockID = SL.StockID, @QuantityToDelete
= SL.Quantity
    FROM Stock_Log SL
    WHERE SL.StLogID = @StLogID;
    UPDATE Stock
    SET Quantity = Quantity - @QuantityToDelete
    WHERE StockID = @StockID;
    DELETE FROM Stock_Log
    WHERE StLogID = @StLogID;

    COMMIT;
END

```

▪ **Procedure : InsertCustomer**

```

CREATE PROCEDURE InsertCustomer
    @CustomerID INT,
    @Name VARCHAR(255),
    @Reg_Date DATE,
    @Address VARCHAR(255),
    @UserID INT
AS
BEGIN
    BEGIN TRANSACTION;

    DECLARE @ExistingCustomer INT;
    SELECT @ExistingCustomer = COUNT(*)
    FROM Customer
    WHERE CustomerID = @CustomerID;

    IF @ExistingCustomer > 0
    BEGIN
        ROLLBACK;
        THROW 51000, 'Customer with the same ID already exists. Insertion
aborted.', 1;
        RETURN;
    END

    INSERT INTO Customer (CustomerID, Name, Reg_Date, Address, UserID)
    VALUES (@CustomerID, @Name, @Reg_Date, @Address, @UserID);

    COMMIT;
END;

```

▪ **Procedure : InsertOrderAndOrderStock**

```

CREATE PROCEDURE InsertOrderAndOrderStock
    @UserID INT,
    @Date DATE,
    @Quantity INT,
    @CustomerID INT,
    @CategoryName VARCHAR(255)
AS
BEGIN
    DECLARE @OrderID INT;
    DECLARE @OrderStockID INT;
    DECLARE @StockID INT;
    DECLARE @UnitPrice DECIMAL(10, 2);
    DECLARE @TotalAmount DECIMAL(10, 2);
    DECLARE @CategoryID INT;

    BEGIN TRANSACTION;

    BEGIN TRY

        SELECT @CategoryID = CategoryID
        FROM Category
        WHERE CategoryName = @CategoryName;

        IF @CategoryID IS NOT NULL
        BEGIN

            SELECT @UnitPrice = UnitPrice
            FROM Category
            WHERE CategoryID = @CategoryID;

            IF @UnitPrice IS NOT NULL
            BEGIN

                SET @TotalAmount = @UnitPrice * @Quantity;

                IF NOT EXISTS (SELECT 1 FROM Category WHERE CategoryID =
@CategoryID)
                BEGIN

                    ROLLBACK;
                    PRINT 'Selected category does not exist. Order and
OrderStock insertion failed.';
                END
            ELSE
            BEGIN

                SELECT @StockID = StockID
                FROM Stock
                WHERE CategoryID = @CategoryID;

                IF @StockID IS NOT NULL AND (SELECT Quantity FROM Stock
WHERE StockID = @StockID) >= @Quantity
                BEGIN

                    INSERT INTO Orders (UserID, Date, Quantity,
TotalAmount, CustomerID, CategoryID)

```

```

VALUES (@UserID, @Date, @Quantity, @TotalAmount,
@CustomerID, @CategoryID);

SET @OrderID = SCOPE_IDENTITY();

INSERT INTO OrderStock (OrderID, StockID)
VALUES (@OrderID, @StockID);

UPDATE Stock
SET Quantity = Quantity - @Quantity
WHERE StockID = @StockID;

SET @OrderStockID = SCOPE_IDENTITY();

COMMIT;

PRINT 'Order and OrderStock inserted successfully.
OrderID: ' + CAST(@OrderID AS VARCHAR(10)) + ', OrderStockID: ' +
CAST(@OrderStockID AS VARCHAR(10));
END
ELSE
BEGIN
ROLLBACK;
PRINT 'Not enough quantity in Stock for CategoryID: ' +
CAST(@CategoryID AS VARCHAR(10)) + '. Order and OrderStock insertion failed.';
END
END
ELSE
BEGIN
ROLLBACK;
PRINT 'UnitPrice not found for CategoryID: ' + CAST(@CategoryID
AS VARCHAR(10)) + '. Order and OrderStock insertion failed.';
END
END
ELSE
BEGIN
ROLLBACK;
PRINT 'CategoryID not found for CategoryName: ' + @CategoryName +
'. Order and OrderStock insertion failed.';
END
END TRY
BEGIN CATCH

ROLLBACK;
PRINT 'An error occurred. Order and OrderStock insertion failed.';
END CATCH
END;

```

User Interfaces

Application Details

- Our standalone application for stock and order management utilizes cutting-edge technology.

Area	Technologies used
Language	C#
Framework	.Net Framework
Database	SQL Server Management studio
Design Toolkit	Visual Studio Guna Framework

Standalone Application Credentials:

- “Role” decides whether the user should be redirected to the User dashboard or Super Admin dashboard. (“User” can be either “Super Admin” or a “Staff-User”)
- So, when we develop the system we get access to super admin for administrative purposes like viewing analytical data, accessing user login history, and managing users.
- On the other hand, get access to normal users for other works like creating orders, managing customers and suppliers managing the water bottle category, and managing the stock log.

User Type (Role)	Username	Password
User	Isuru	123
Super Admin	Roshan	123

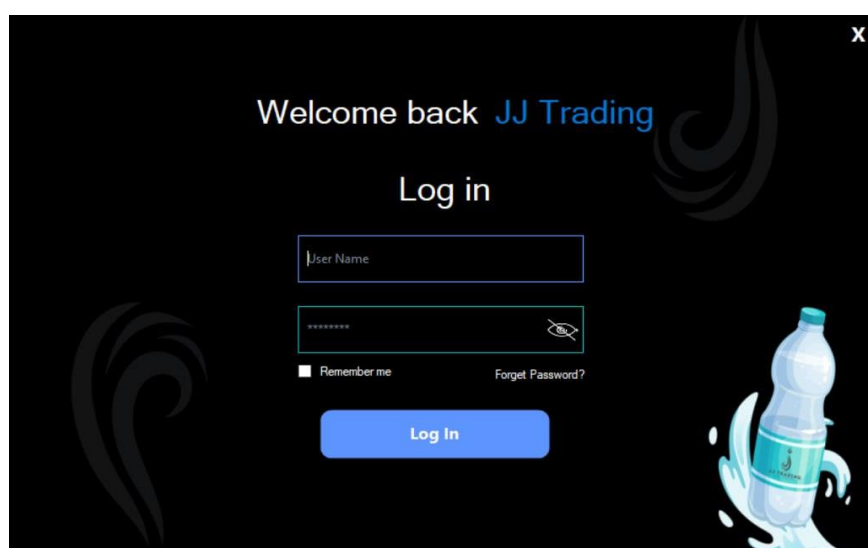
Screenshots of the Application



Lording Screen



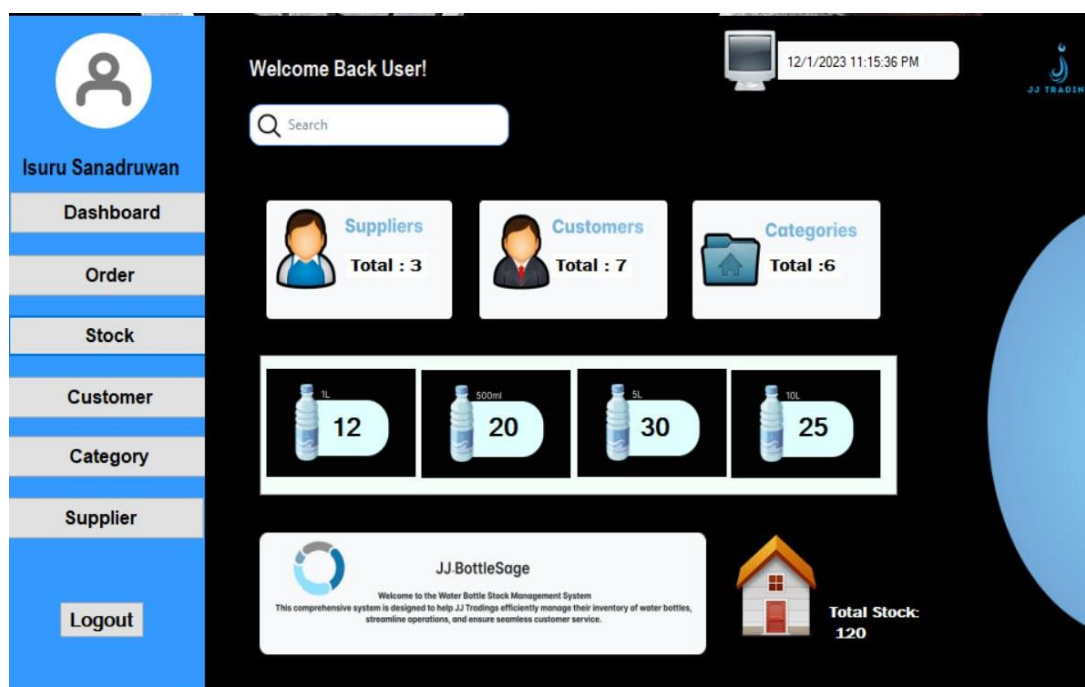
Login page



- “Role” decides whether the user should be redirected to the User dashboard or Super Admin dashboard. (“User” can be either “**Super Admin**” or a “**Staff-User**”)
- So, when we develop the system we get access to super admin for administrative purposes like viewing analytical data, accessing user login history, and managing users.
- On the other hand, get access to normal users for other works like creating orders, managing customers and suppliers managing the water bottle category, and managing the stock log.

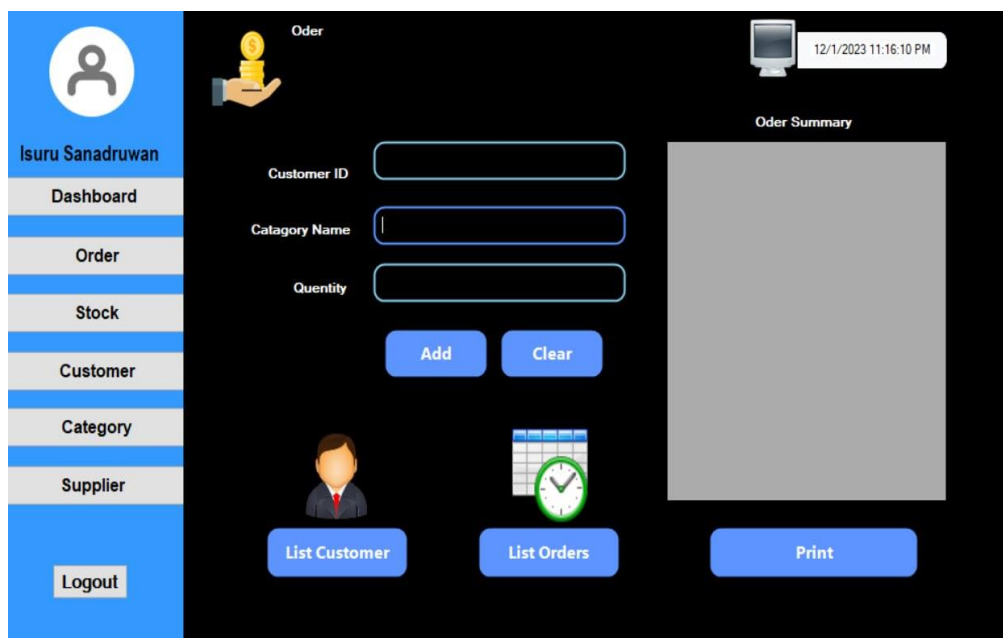
User Dashboard

- This is the Staff-User (normal user) dashboard.
- In this screen, the user can see the total number of suppliers, customers, categories, and total stock available.
- Automatically generated the date and time of this system.
- In this screen, the user can see the total number of available categories separately.



Order Screen

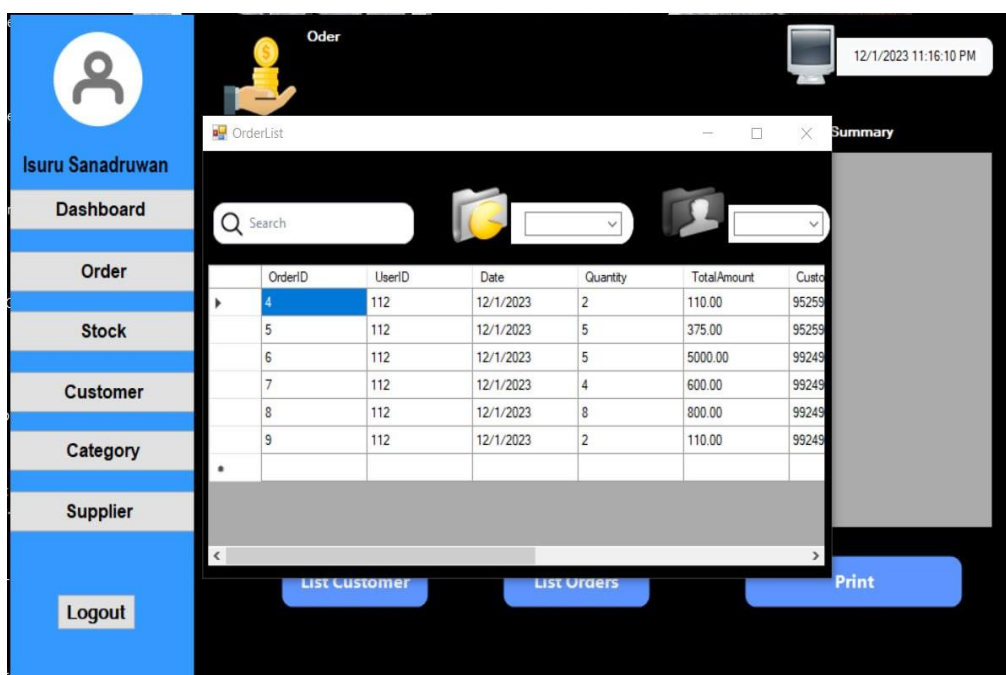
- The user can add a water bottle order by providing a customer ID, Category Name, and Quantity to the system.
- The user can see the list of customers and list orders.
- The user can print the order summary using this system.



The screenshot shows the 'Order' screen of the application. On the left is a sidebar with the user's name 'Isuru Sanadruwan' and a menu with options: Dashboard, Order, Stock, Customer, Category, Supplier, and Logout. The main area is titled 'Oder' (sic) and contains three input fields: 'Customer ID', 'Category Name', and 'Quantity'. Below these fields are 'Add' and 'Clear' buttons. At the bottom of the main area are three buttons: 'List Customer' (with a person icon), 'List Orders' (with a calendar icon), and 'Print' (with a printer icon). On the right side, there is a section titled 'Oder Summary' (sic) which is currently empty. The top right corner shows the date and time: '12/1/2023 11:16:10 PM'.

Oder List Screen

- In this screen, the user can see all the records of the water bottle orders.



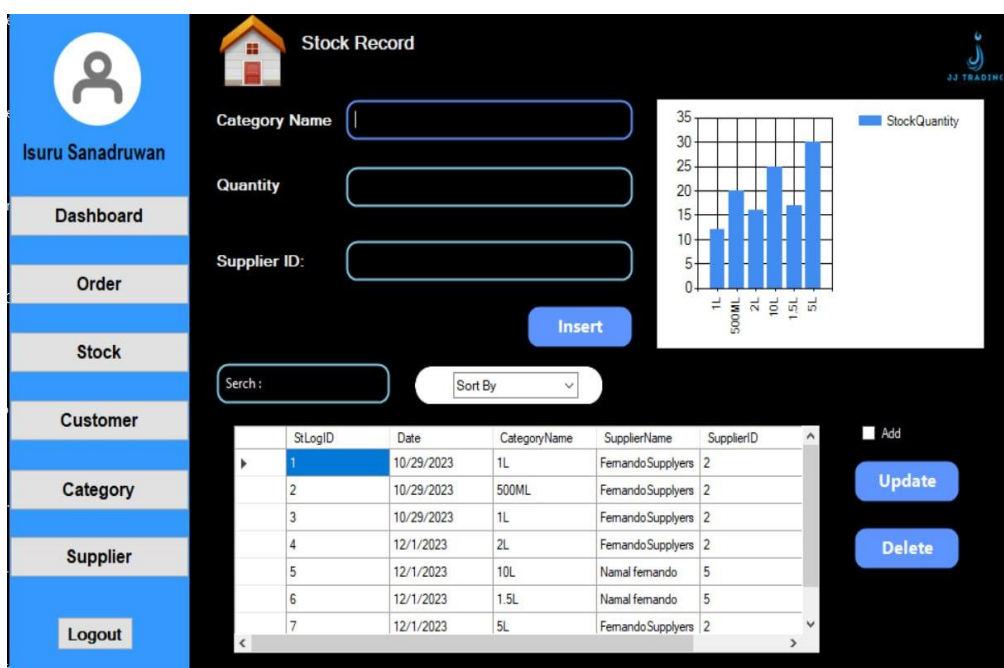
The screenshot shows the 'Oder List' screen. The sidebar is identical to the previous screen. The main area is titled 'Oder' and contains a search bar, a folder icon, and a dropdown menu. Below these is a table with the following data:

OrderID	UserID	Date	Quantity	TotalAmount	Custo
4	112	12/1/2023	2	110.00	95259
5	112	12/1/2023	5	375.00	95259
6	112	12/1/2023	5	5000.00	99249
7	112	12/1/2023	4	600.00	99249
8	112	12/1/2023	8	800.00	99249
9	112	12/1/2023	2	110.00	99249

Below the table are three buttons: 'List Customer', 'List Orders', and 'Print'. On the right side, there is a section titled 'Summary' which is currently empty. The top right corner shows the date and time: '12/1/2023 11:16:10 PM'.

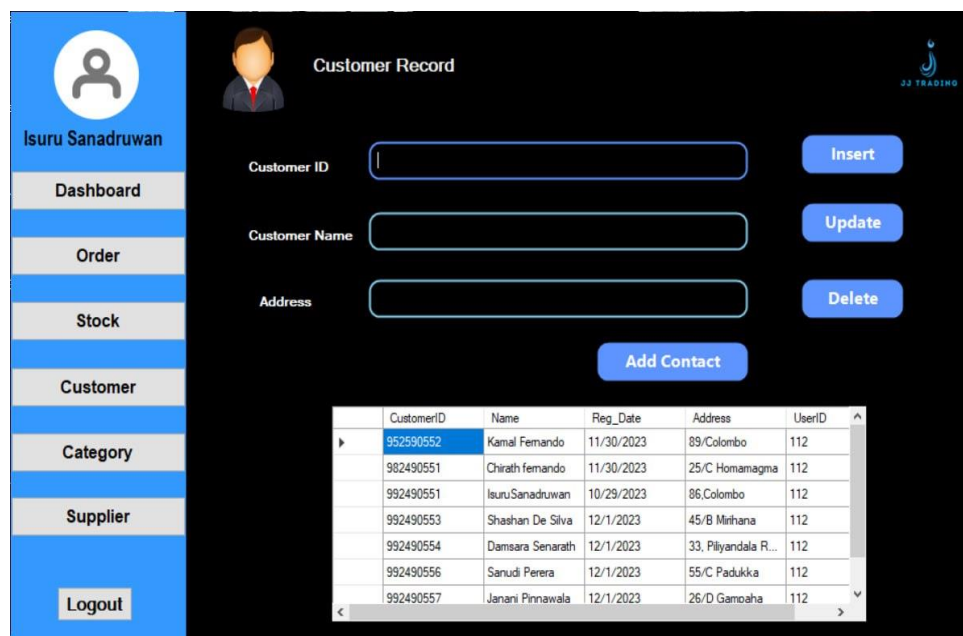
Stock Screen

- The user can add a water bottle stock by providing a Category Name, Quantity, and Supplier ID to the system and the User can see the available water bottle stock quantity using the Bar Chart.
- The User can search the stocks and update and delete stock.
- Using this screen, The User can see all the records like StLogID, Date, Category Name, Supplier Name, and Supplier ID.



Customer Screen

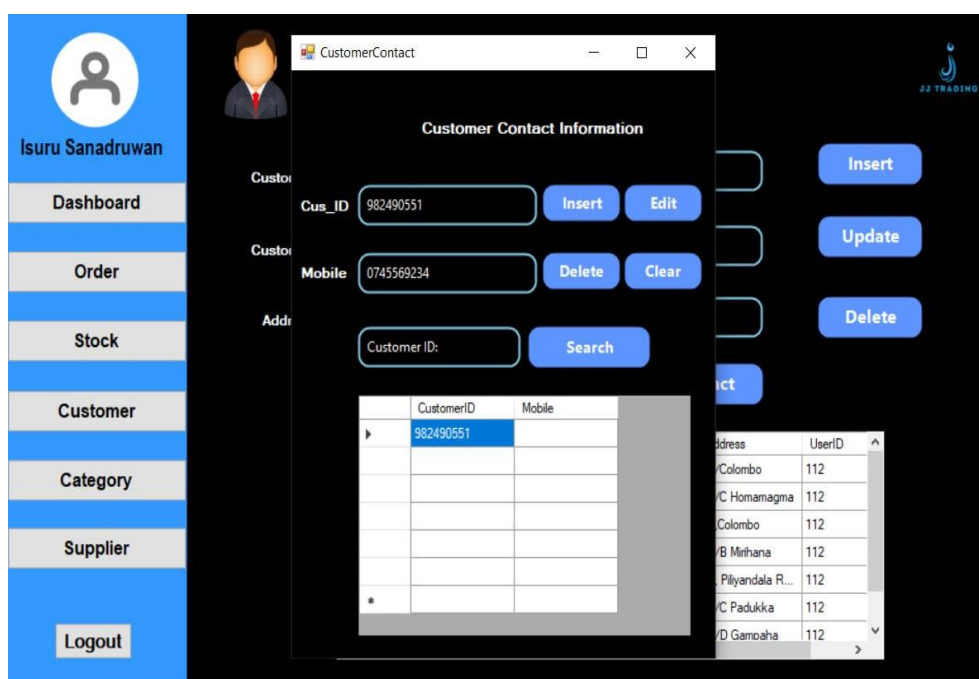
- The user can add customers, update customers, and delete customers in this system.
- The user can add customers by providing Customer ID, Customer Name, and Address to this system. (User can manage customers)
- In this screen, the user can see all the records of the customer.



CustomerID	Name	Reg_Date	Address	UserID
952590552	Kamal Fernando	11/30/2023	89/Colombo	112
982490551	Chirath Fernando	11/30/2023	25/C Homamagma	112
992490551	Isuru Sanadruwan	10/29/2023	86/Colombo	112
992490553	Shashan De Silva	12/1/2023	45/B Minihana	112
992490554	Damsara Senarath	12/1/2023	33, Piliyandala R...	112
992490556	Sanudi Perera	12/1/2023	55/C Padukka	112
992490557	Janani Pinnawala	12/1/2023	26/D Gamuaha	112

Customer Contact

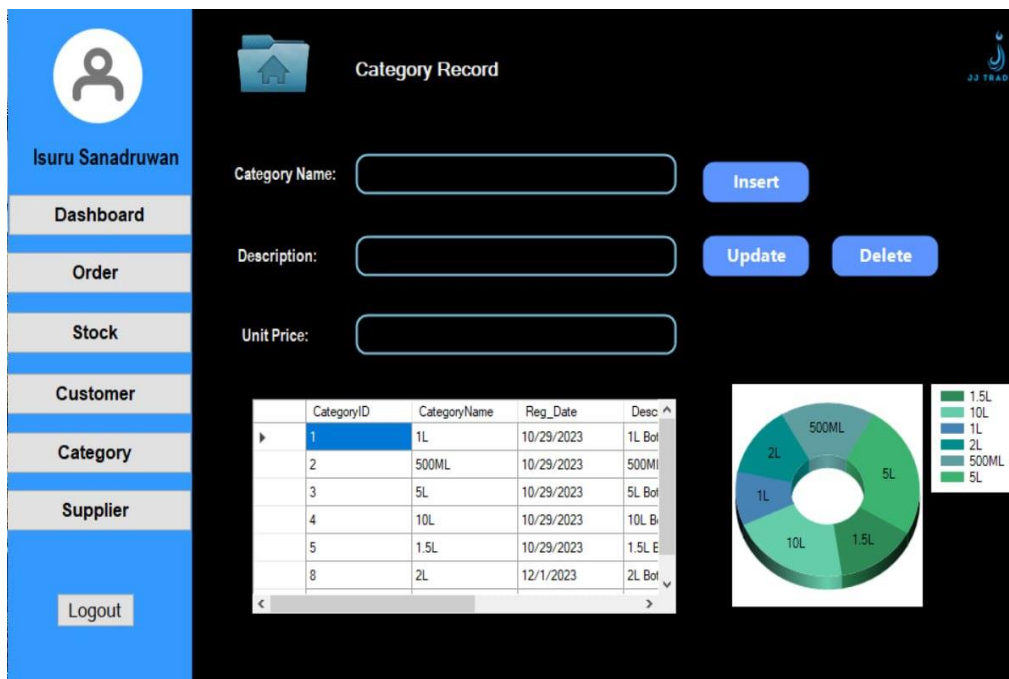
- User can add, modify, or delete customer contact information.



CustomerID	Mobile
982490551	

Category Screen

- The user can add a category by providing a Category Name, Description and Unit Price to the system and the User can see the available categories using the Pie Chart.
- The User can update and delete category.
- Using this screen, The User can see all the records like Category ID, Category Name, Registration Date and Description.



Category Record

Category Name: **Insert**

Description: **Update** **Delete**

Unit Price:

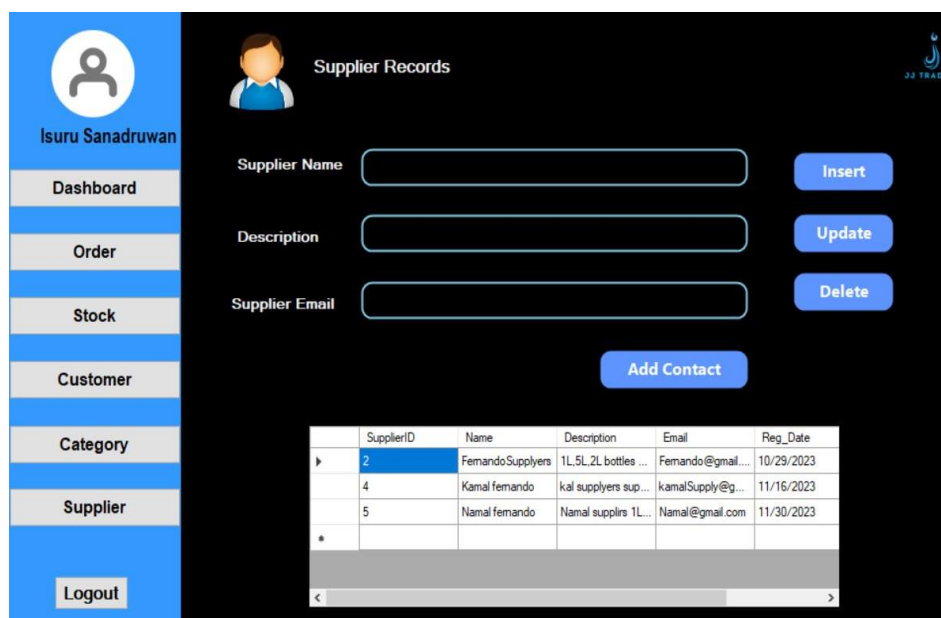
CategoryID	CategoryName	Reg_Date	Desc
1	1L	10/29/2023	1L Bot
2	500ML	10/29/2023	500MI
3	5L	10/29/2023	5L Bot
4	10L	10/29/2023	10L B
5	1.5L	10/29/2023	1.5L E
8	2L	12/1/2023	2L Bot

Donut Chart Legend:

- 1.5L
- 10L
- 1L
- 2L
- 500ML
- 5L

Supplier Screen

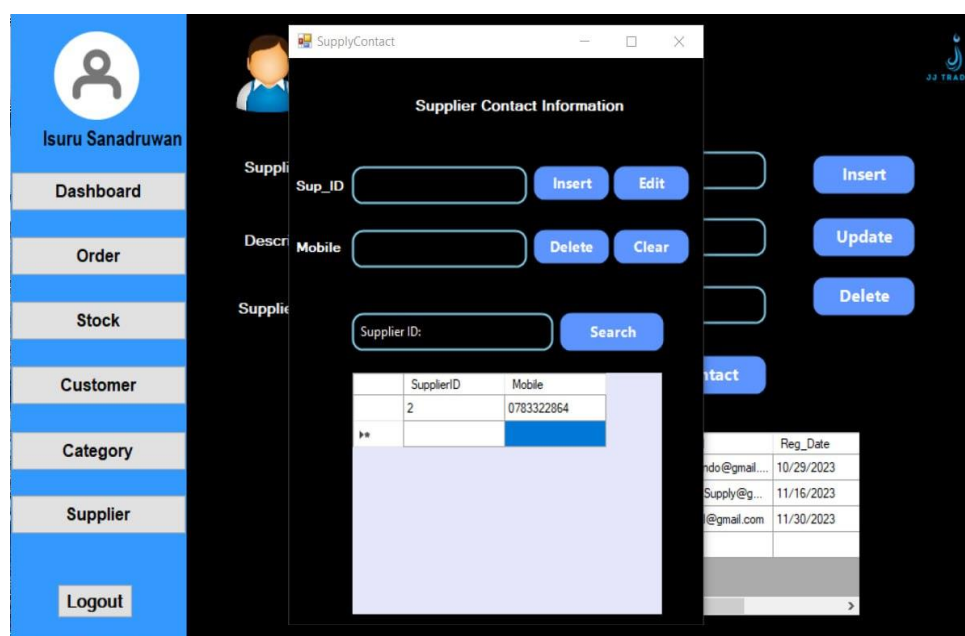
- The user can add suppliers, update suppliers, and delete suppliers in this system.(User can manage suppliers)
- The user can add suppliers by providing Supplier Name, Description, and Supplier Email to this system.
- In this screen, the user can see all the records of the supplier.



SupplierID	Name	Description	Email	Reg_Date
2	FemandoSuppliers	1L,5L,2L bottles ...	Femando@gmail...	10/29/2023
4	Kamal femando	kal suppliers sup...	kamalSupply@g...	11/16/2023
5	Namal femando	Namal supplrs 1L...	Namal@gmail.com	11/30/2023

Supplier Contact Screen

- User can add, modify, or delete supplier contact information.



SupplierID	Mobile
2	0783322864

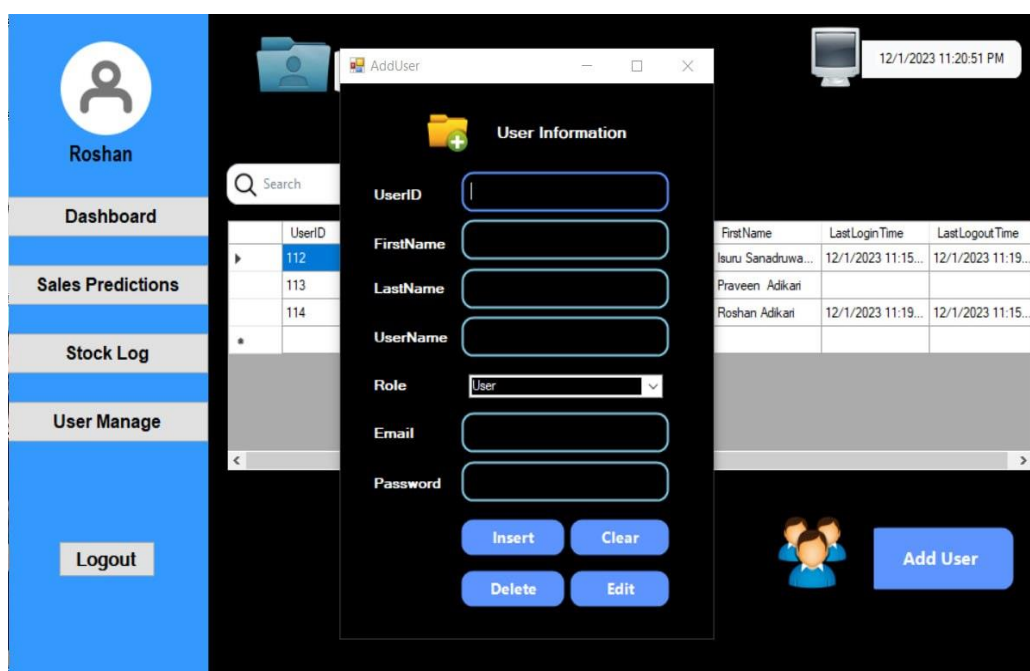
Super Admin Dashboard

- This is the Super Admin dashboard.
- In this screen, the super admin can see the total number of suppliers, customers, and categories.
- In this screen, the user can see the total number of available categories separately using a pie chart.



Add User Screen

- Super Admin can add user information.



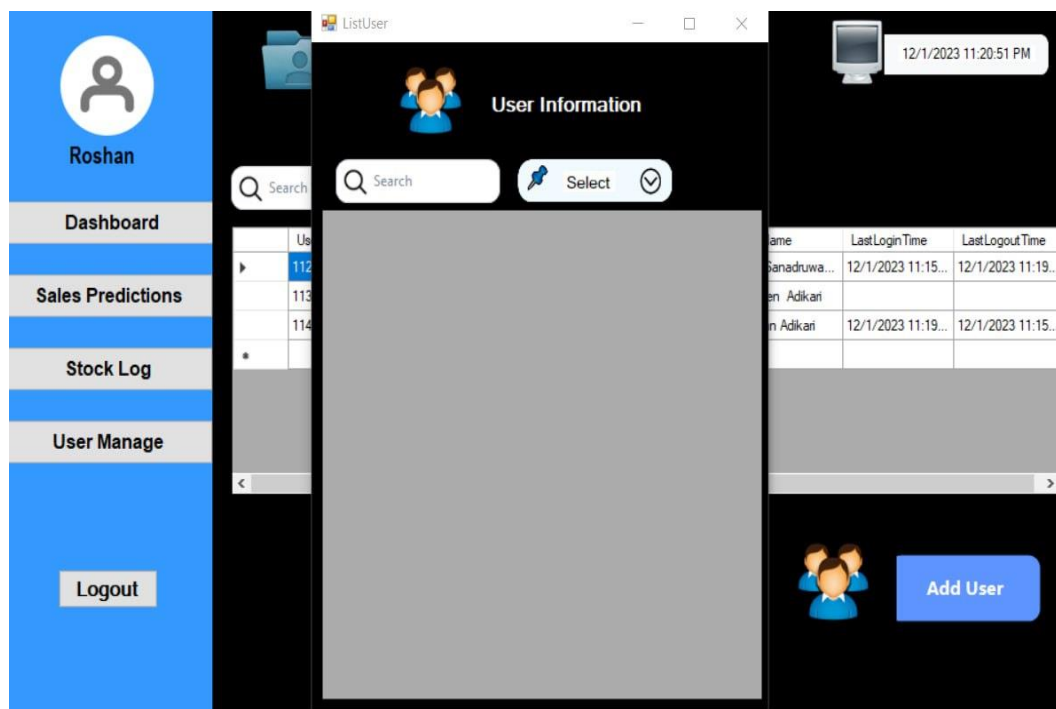
The screenshot shows the 'Add User' screen. It includes a sidebar with navigation links: Dashboard, Sales Predictions, Stock Log, User Manage, and Logout. The main content area displays 'Add User' and the date '12/1/2023 11:20:51 PM'. A modal window titled 'User Information' is open, showing fields for UserID, FirstName, LastName, UserName, Role (set to 'User'), Email, and Password. Below the fields are buttons for Insert, Clear, Delete, and Edit. In the background, there is a table with user information:

First Name	Last Login Time	Last Logout Time
Isuru Sanadruwa...	12/1/2023 11:15...	12/1/2023 11:19...
Praveen Adikari		
Roshan Adikari	12/1/2023 11:19...	12/1/2023 11:15...

At the bottom right, there is an 'Add User' button.

List User Screen

- Super Admin can list user information.



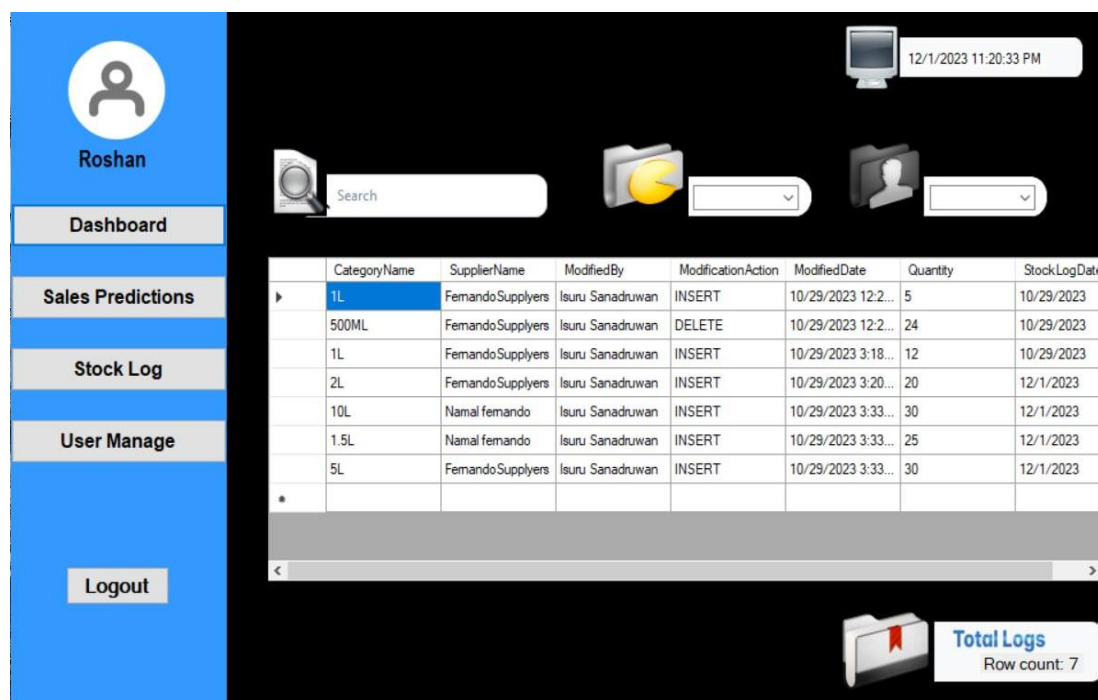
Sales Prediction Screen

- Super Admin can see all sales statistics of the JJ Trading company. Using these analysis views the super admin predicts the sales for the growth of the company.



Stock Log Screen

- Super Admin can view the stock log records and see the total logs of JJ Trading.

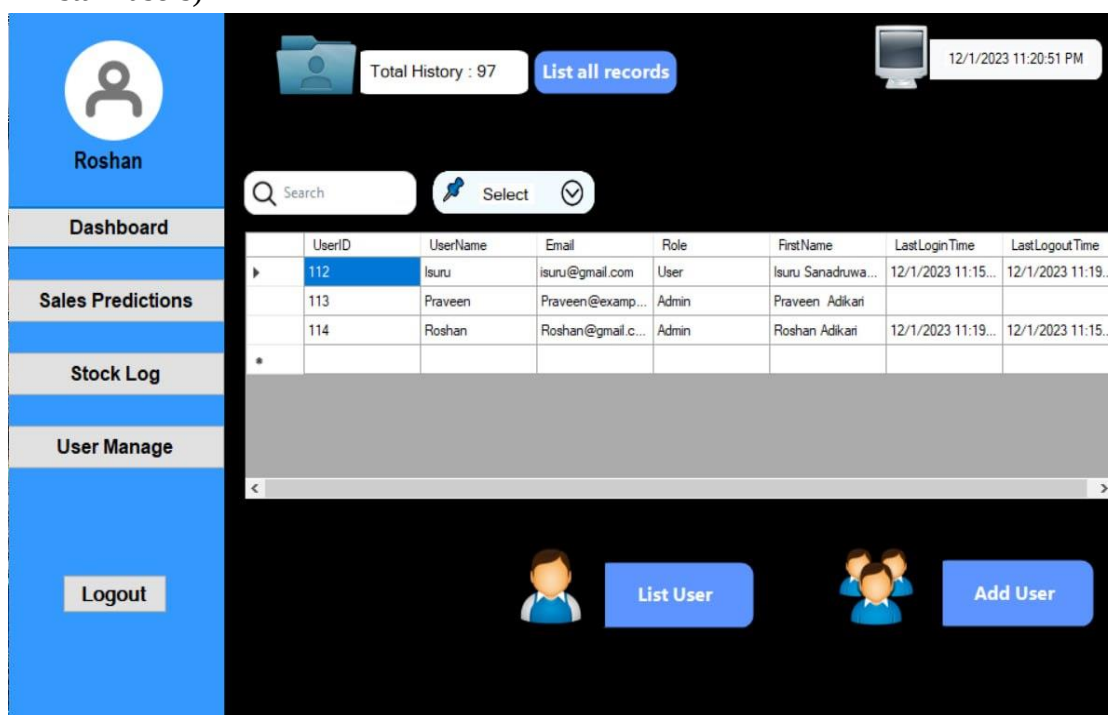


The screenshot shows the Stock Log interface. On the left is a blue sidebar with a user profile for 'Roshan' and navigation buttons: Dashboard, Sales Predictions, Stock Log (highlighted), User Manage, and Logout. The main area has a dark background. At the top right, it shows the date and time: 12/1/2023 11:20:33 PM. Below this are search and filter icons. A table displays stock log records with columns: CategoryName, SupplierName, ModifiedBy, ModificationAction, ModifiedDate, Quantity, and StockLogDate. The table contains 7 rows of data. At the bottom right, there is a 'Total Logs' box showing 'Row count: 7'.

CategoryName	SupplierName	ModifiedBy	ModificationAction	ModifiedDate	Quantity	StockLogDate
1L	FernandoSuppliers	Isuru Sanadruwan	INSERT	10/29/2023 12:2...	5	10/29/2023
500ML	FernandoSuppliers	Isuru Sanadruwan	DELETE	10/29/2023 12:2...	24	10/29/2023
1L	FernandoSuppliers	Isuru Sanadruwan	INSERT	10/29/2023 3:18...	12	10/29/2023
2L	FernandoSuppliers	Isuru Sanadruwan	INSERT	10/29/2023 3:20...	20	12/1/2023
10L	Namal fernando	Isuru Sanadruwan	INSERT	10/29/2023 3:33...	30	12/1/2023
1.5L	Namal fernando	Isuru Sanadruwan	INSERT	10/29/2023 3:33...	25	12/1/2023
5L	FernandoSuppliers	Isuru Sanadruwan	INSERT	10/29/2023 3:33...	30	12/1/2023

User Manage

- Super admin can see the last login time and last logout time for administrative purposes.
- Super admin can list all the user records of the system. (Super Admin can manager staff-users)



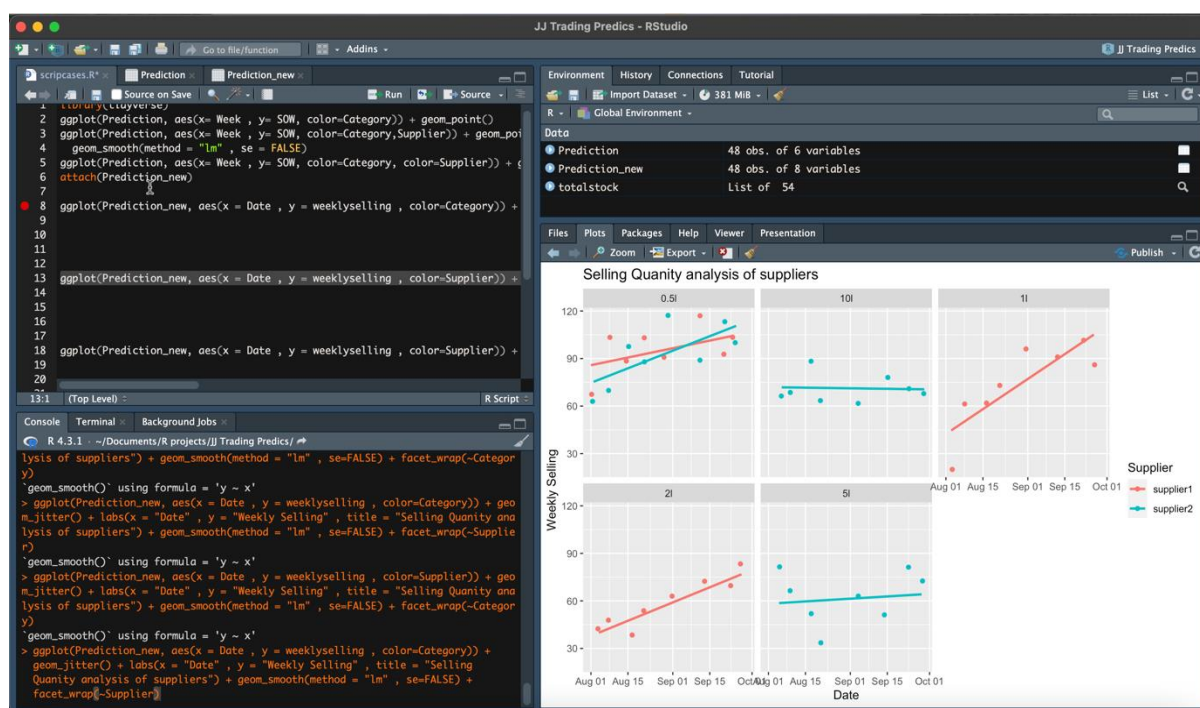
The screenshot shows the User Manage interface. On the left is a blue sidebar with a user profile for 'Roshan' and navigation buttons: Dashboard, Sales Predictions, Stock Log, User Manage (highlighted), and Logout. The main area has a dark background. At the top right, it shows the date and time: 12/1/2023 11:20:51 PM. Below this are search and select icons. A table displays user records with columns: UserID, UserName, Email, Role, FirstName, LastLoginTime, and LastLogoutTime. The table contains 3 rows of data. At the bottom, there are buttons for 'List User' and 'Add User'.

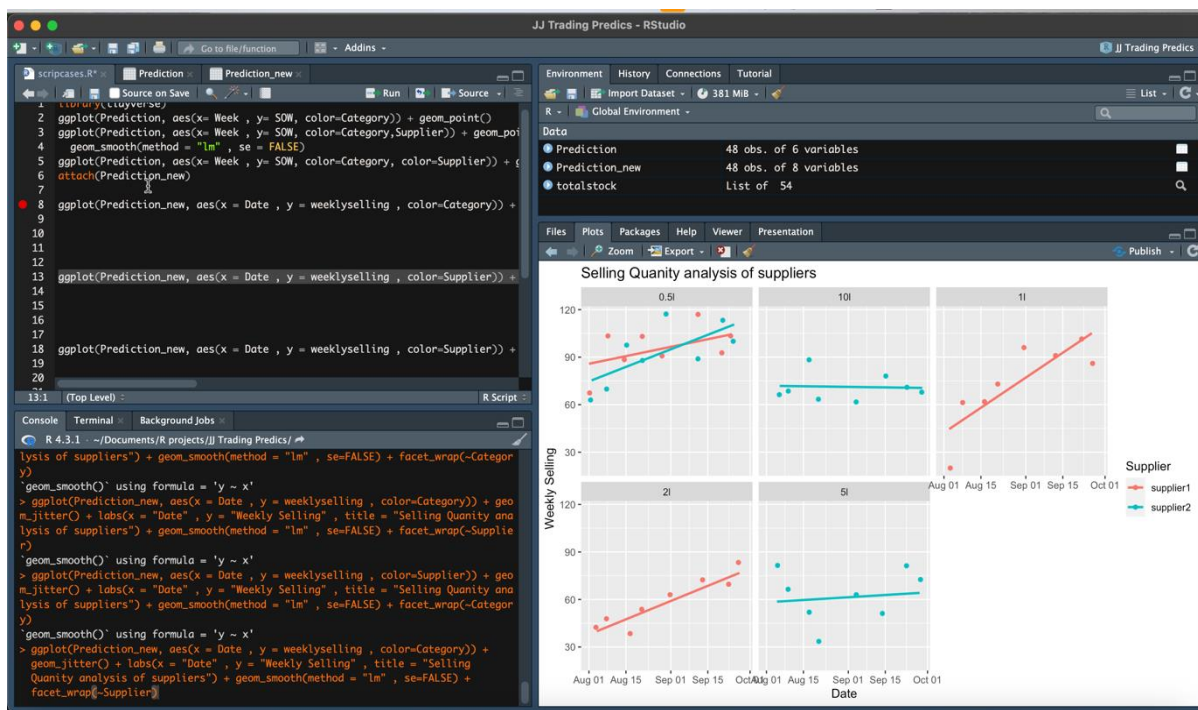
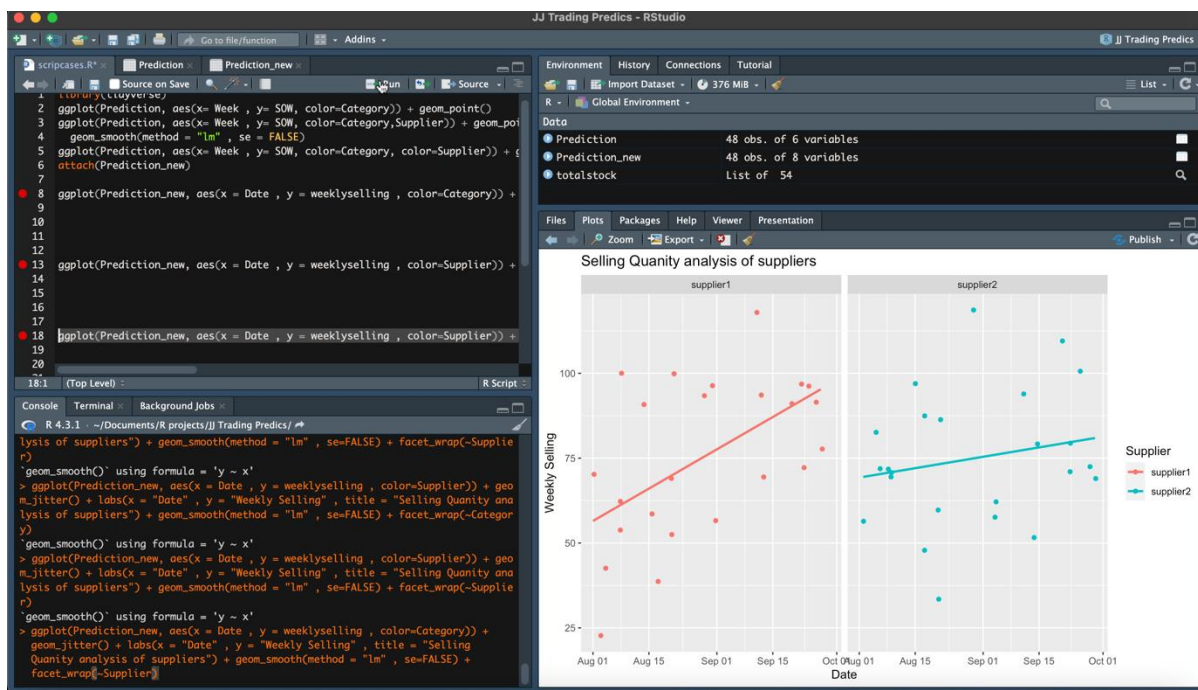
UserID	UserName	Email	Role	FirstName	LastLoginTime	LastLogoutTime
112	Isuru	isuru@gmail.com	User	Isuru Sanadruwa...	12/1/2023 11:15...	12/1/2023 11:19...
113	Praveen	Praveen@examp...	Admin	Praveen Adikari		
114	Roshan	Roshan@gmail.c...	Admin	Roshan Adikari	12/1/2023 11:19...	12/1/2023 11:15...

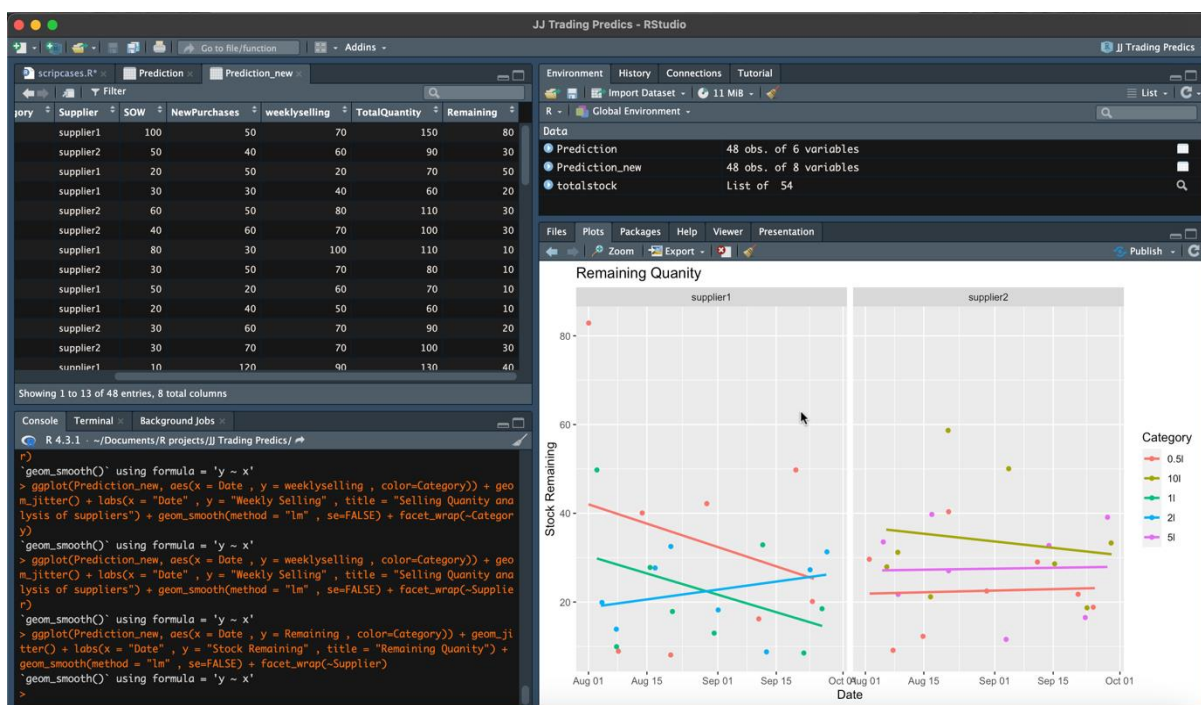
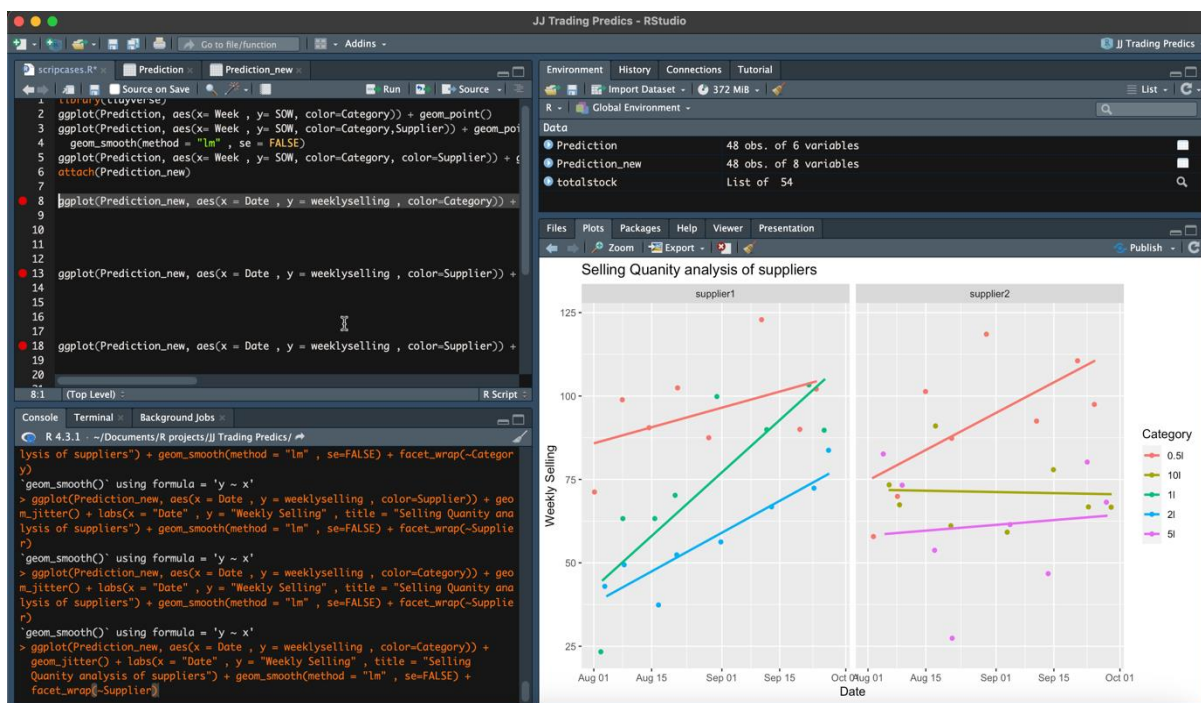
Analysis and Future Prediction

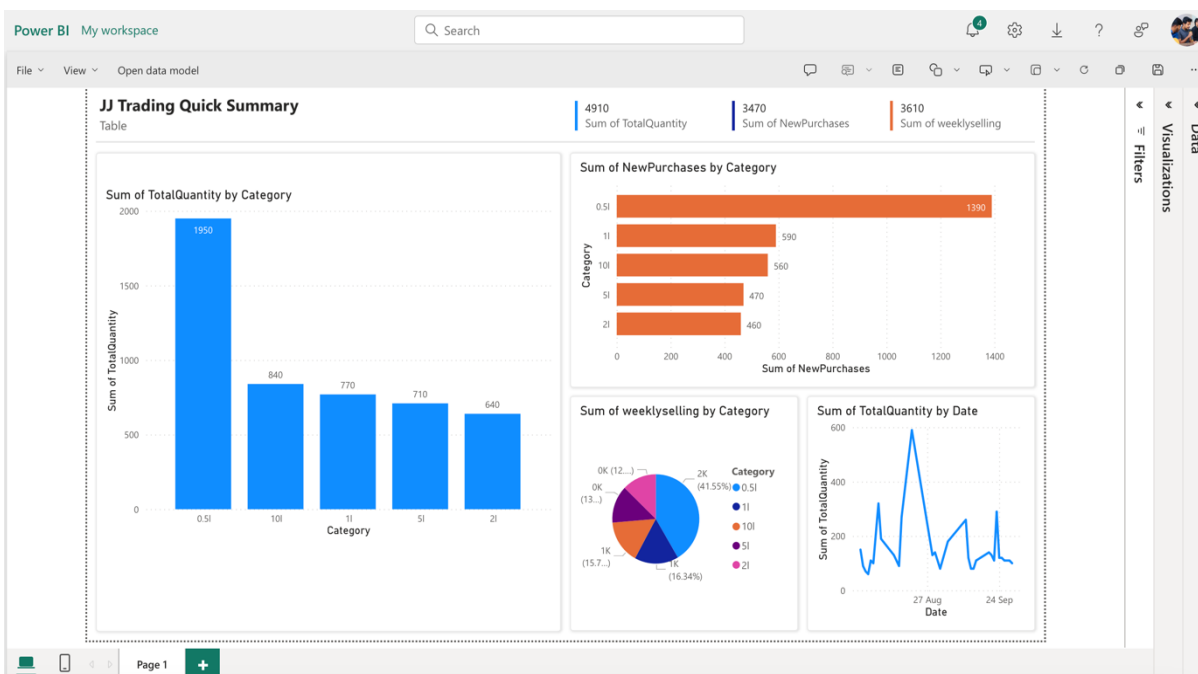
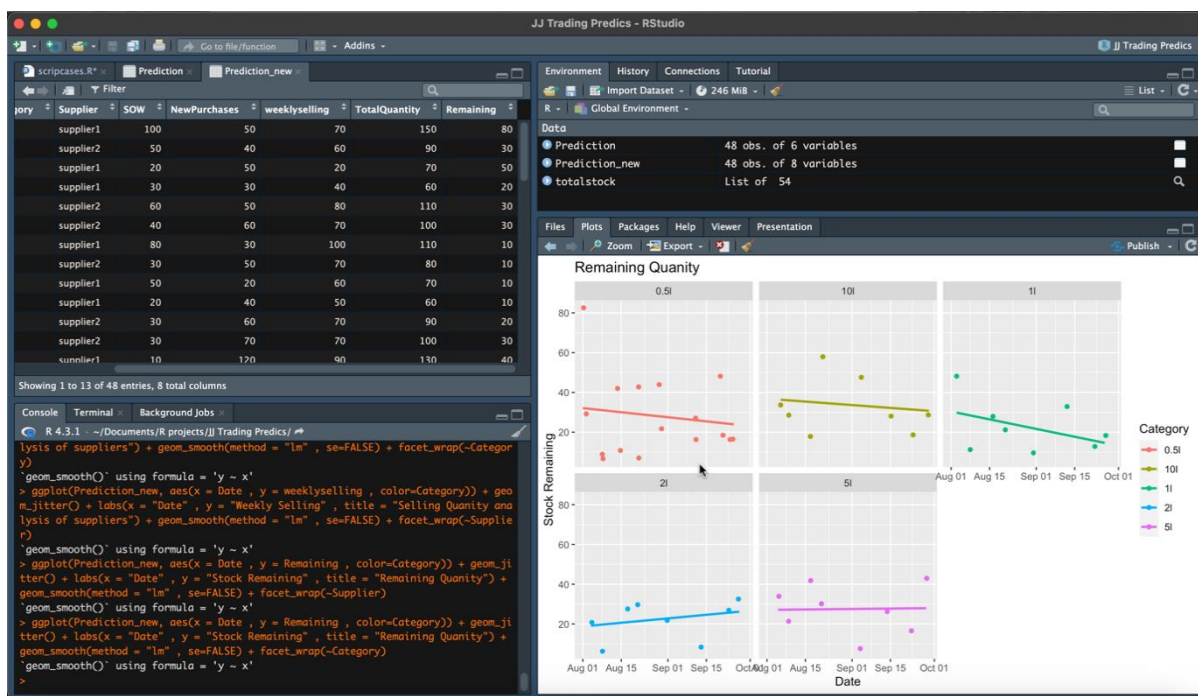
- The supply and selling quantity (stock) analysis diagram in an analytical report is a visual representation (used **R Tool**) that provides a concise overview of the relationship between the quantity of products supplied and the quantity sold.
- These below diagrams typically showcase trends and patterns in the supply and demand dynamics.
- It will help stakeholders of “JJ Trading” quickly grasp the efficiency of the supply chain and the market's responsiveness to products, making it a valuable tool for decision-makers to optimize operations and strategic planning.

Selling Quantity Analysis Diagrams:







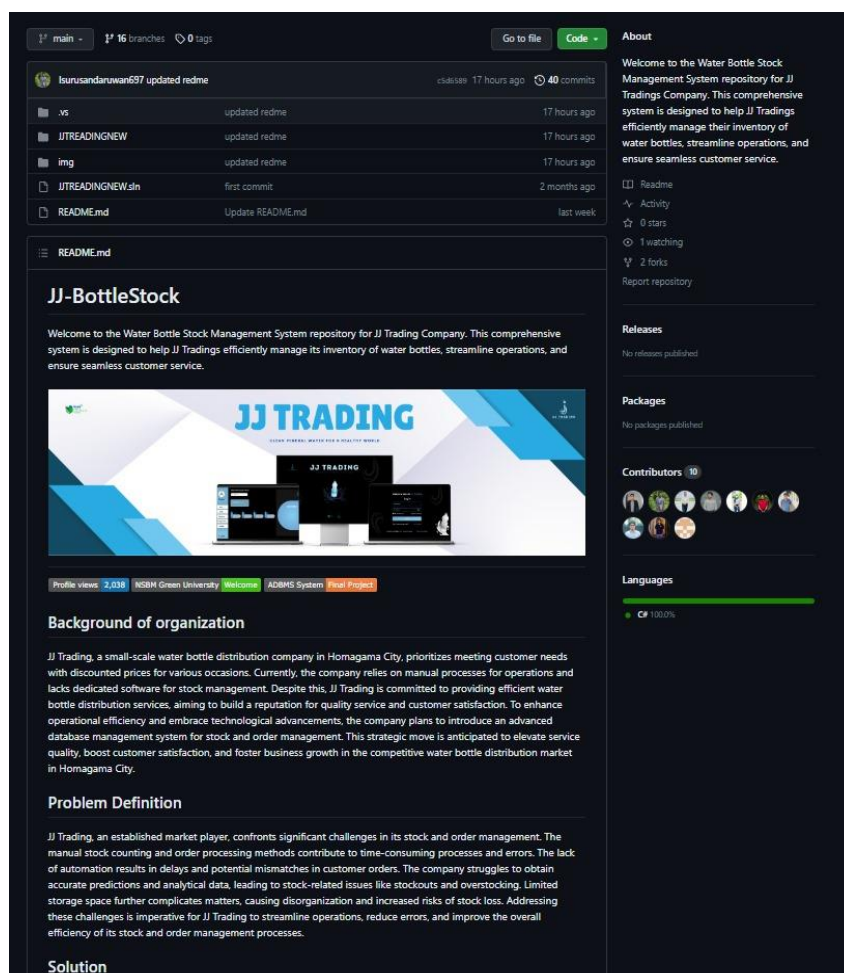


- The above diagrams provide a concise visual summary of water bottle sales data, showing water bottle categories, sales volume and comparative performance which enables quick insights for decision-makers at “JJ Trading”.

GitHub Repository and Backup File Link

- **Repository link :**

<https://github.com/Isurusandaruwan697/JJ-BottleStock>



- **Backup File Drive Link :**

https://drive.google.com/file/d/1IsZqRRzYUQnilO4Bwi5fmIq1Mk3IWdsd/view?usp=drive_link

Conclusion

In conclusion, the implementation of an advanced stock and order management system at JJ Trading represents a pivotal step towards improving operational efficiency and service quality. This project solves problems that have hampered JJ Trading's expansion and customer satisfaction, such as manual stock counts, restricted stock logging, order processing difficulties, and data inaccuracies.

By transitioning to an automated system, JJ Trading will streamline stock management, enhance order processing accuracy, and meet customer demands promptly. Comprehensive stock logging will provide historical insights for trend analysis and supplier assessment. Data security measures will protect important information, instilling trust among stakeholders.

With the JJBottleStage solution, they can anticipate business growth and a strong presence in the competitive water bottle distribution market in Homagama City.

****End of the report****